

ORIGIN Frame: Static Assurance for Frames, Epochs, and Covariance in Scientific Python

Design Evolution, Adversarial Evaluation, and Final Research Proposal

Concept, Formal Proposal, and Critical Assessment

August 19, 2026

Abstract

Scientific Python frequently represents state vectors, coordinates, epochs, and covariances as ordinary numbers and arrays. Runtime libraries provide essential transformations, but physical semantics can disappear between library and function boundaries. This paper ultimately proposes ORIGIN Frame, a gradual static analysis system for coordinate-frame, epoch/time-scale, representation, and covariance consistency in NumPy-, Astropy-, and SPICE-oriented Python. The proposal does not claim invention of frame typing: PHYSFRAME and compile-time geometry libraries provide direct prior art. Its potential contribution is a Python scientific-array formulation that treats runtime-dependent epochs and covariance transformation as first-class obligations, accompanied by a formal core, practical checker, and independently curated space-science fault benchmark. The paper also documents and rejects broader provenance, release-gate, and general physical-type concepts. The final recommendation is intentionally narrow and remains conditional on evidence that existing systems do not already solve the identified user need.

Contents

1	Vision	7
2	Problem Statement	7
3	Core Abstractions	7
3.1	Artifact	7
3.2	Step	8
3.3	Claim	8
3.4	Evidence capsule	8
4	Formal Model	8

4.1	Scientific acceptance	9
5	Proposed User Interface	10
6	Minimal Software Architecture	10
7	Non-Negotiable Engineering Invariants	11
8	Applications in Space Science and AI	11
8.1	Astronomical data-product certification	11
8.2	AI decision evidence	12
8.3	Orbit determination and simulation	12
8.4	Calibration impact analysis	12
8.5	Candidate flagship study	12
9	Verification and Testing Strategy	12
10	Research Questions and Experimental Design	13
11	Security, Privacy, and Assurance	13
12	Adversarial Evaluation of the Concept	14
12.1	Summary judgment	14
12.2	Complete inventory of strengths	14
12.3	Mathematical and semantic weaknesses	16
12.4	Scientific weaknesses	17
12.5	Engineering weaknesses	18
12.6	Usability and incentive weaknesses	19
12.7	Why the library might not be useful	19
12.8	Why the library might be very useful	20
12.9	Why it may not be a genuine scientific contribution	20
12.10	Why journals might reject the paper	21
12.11	Why NASA may not need it	22

12.12	Why ESA may not need it	23
13	Adversarial Evaluation of the Existing Code	23
13.1	Concrete strengths of the prototype	23
13.2	Demonstrated correctness failures	24
13.3	Structural defects in the prototype	24
13.4	Code-level conclusion	25
14	Revised Research Position	25
15	Final Redesign: ORIGIN Gate	25
15.1	The premise that cannot be engineered away	25
15.2	One-sentence product definition	26
15.3	The concrete real-world problem	26
15.4	Five user-visible objects	27
15.5	Command-line experience	27
15.6	Python interface	28
15.7	Standards-first architecture	28
15.8	Byte identity and semantic fingerprints	29
15.9	Contract outcome logic	29
15.10	Capture coverage rather than absolute completeness	30
15.11	Minimal implementation	31
15.12	Engineering-masterpiece criteria	31
15.13	Easy and unusually strong testing	31
15.14	Flagship astrophysics demonstrator	32
15.15	The genuine research contribution	33
15.16	Global project strategy	33
15.17	A credible agency-adoption funnel	34
15.18	Date-specific execution plan	34
15.19	Quantitative release and research thresholds	35
15.20	Kill criteria	36

15.21	Final strategic conclusion	36
16	Second Red-Team Review: Could ORIGIN Gate Be Worthless?	36
16.1	Brutally concise assessment	37
16.2	The strongest redundancy argument	37
16.3	The four outcomes are useful but probably not novel	38
16.4	The central theorem risks being tautological	38
16.5	Contracts can formalize error instead of eliminating it	39
16.6	The observation boundary may be an honesty label, not evidence	39
16.7	Hashing may add cost without adding useful trust	39
16.8	The statistical layer remains fragile	40
16.9	The FITS flagship may be particularly redundant	40
16.10	Python may be the wrong center of gravity	40
16.11	The independent verifier may not be independent enough	41
16.12	Standards-first can become standards-overload	41
16.13	The release gate can become a bureaucratic bottleneck	41
16.14	Security and privacy can reverse the value proposition	41
16.15	Economic reasons it may have negative value	42
16.16	Why NASA may consider it completely unnecessary	42
16.17	Why ESA may consider it completely unnecessary	43
16.18	Why the concept may not be a scientific contribution	43
16.19	Why journals might reject the current manuscript immediately	43
16.20	Why it might be completely worthless	44
16.21	Why it might nevertheless be unusually useful	45
16.22	Current readiness score	45
16.23	The one experiment that should decide whether to continue	46
17	Final Recommendation: ORIGIN Phys	47
17.1	Why this problem is sharper	47
17.2	What already exists and what must be new	48

17.3	Core typing judgments	48
17.4	Soundness objective	49
17.5	Python representation	50
17.6	Diagnostics as a primary feature	50
17.7	Minimal architecture	51
17.8	Space and astrophysics applications	51
17.9	Why this could be globally useful	52
17.10	Easy and rigorous testing	52
17.11	The research paper	53
17.12	Honest limitations	54
17.13	NASA and ESA path for 2027	54
17.14	Global engineering strategy	55
17.15	Kill criteria for the final pivot	55
17.16	Final verdict	56
18	Adversarial Review of ORIGIN Phys	56
18.1	Summary judgment	56
18.2	Complete inventory of strengths	57
18.3	Prior art and similarity risk	58
18.4	Fundamental type-theoretic difficulties	58
18.5	Python and NumPy implementation problems	60
18.6	Runtime boundary and data-ingestion problems	61
18.7	Usability and adoption weaknesses	61
18.8	Security and assurance limits	61
18.9	Why it might not be useful	62
18.10	Why it might be very useful	62
18.11	Why it might not be a genuine scientific contribution	62
18.12	Why journals might reject it	63
18.13	Why NASA and ESA might not need it	63
18.14	Why NASA and ESA might need it	64

18.15	Why it might be completely worthless	64
18.16	Updated readiness score	65
18.17	The decisive feasibility experiment	65
19	Ultimate Narrowing: ORIGIN Frame	66
19.1	Precisely bounded novelty claim	66
19.2	Minimal semantic model	67
19.3	Illustrative Python interface	67
19.4	Critical implementation decision	68
19.5	The genuine research program	68
19.6	The 10/10 engineering rule	69
19.7	Agency adoption strategy	69
19.8	Go/no-go experiment	70
19.9	Ultimate conclusion	70
20	Publication and Adoption Strategy	71
21	Roadmap to a 2027 Pilot	71
22	Success Criteria	71

1 Vision

The guiding principle is:

Two numerical values with incompatible physical meanings should not be indistinguishable to scientific-development tools.

The final proposal is a gradual frame-and-epoch checker for ordinary Python and NumPy code. It seeks early detection of incompatible coordinate frames, epochs, time scales, representations, and covariance relationships without replacing scientific runtime objects or reimplementing unit systems.

The original ORIGIN prototype instead attempted transparent provenance by wrapping arbitrary values and intercepting operations. A later design proposed an evidence-contract release gate. Both ideas are retained below as design history and adversarial evidence: the proxy was unsound, while the gate risked duplicating mature validation and provenance systems.

Design-evolution note. The next sections preserve the intermediate ORIGIN Gate proposal and its adversarial evaluation because they document why the provenance-centered direction was rejected. The later section titled “Ultimate Narrowing: ORIGIN Frame” supersedes both the intermediate gate and the broader ORIGIN Phys proposal.

2 Problem Statement

For a derived result, an investigator should be able to answer five questions:

1. What data, software, model, parameters, and environment produced it?
2. Are those components unchanged since the result was created?
3. Can the computation be replayed independently?
4. If exact replay differs, is the difference scientifically acceptable?
5. Which downstream claims become invalid when one dependency changes?

Containers, lockfiles, workflow graphs, experiment trackers, checksums, and data catalogues each solve part of this problem. They do not by themselves provide a portable and executable scientific validity contract. ORIGIN is intended to compose with these systems rather than replace them.

3 Core Abstractions

3.1 Artifact

An artifact is an immutable description of an input, output, model, calibration file, table, array, message, or other scientific object. It contains a type and schema, a content identifier, optional external location, and domain metadata such as units, coordinate frame, observation identifier, or model version. Large data remain in their existing archive or object store; ORIGIN streams their digest without copying them into the capsule.

3.2 Step

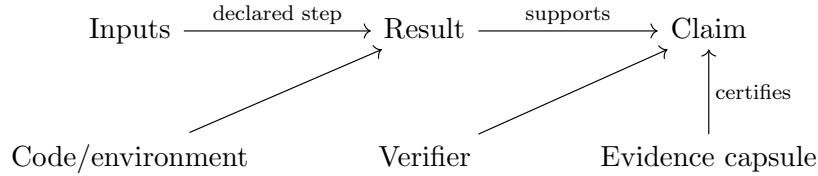
A step is a recorded execution with declared inputs and outputs. Its record may include the code digest, parameters, dependency environment, random state, numerical backend, hardware-relevant attributes, timing, warnings, status, and failure information. A failed step is evidence and must not disappear from the record.

3.3 Claim

A claim is a scientific or engineering assertion supported by one or more artifacts and steps. Examples include “this source is a transit candidate,” “this telemetry interval is anomalous,” and “this calibrated image satisfies the release quality criteria.” A claim includes an executable verifier and an explicit policy describing the conditions under which it remains valid.

3.4 Evidence capsule

An evidence capsule is the portable representation of the relevant subgraph, validation policy, and integrity commitments. It should be readable without importing ORIGIN and verifiable by a separately implemented command-line tool.



4 Formal Model

Let an evidence record contain a finite, versioned event graph

$$G = (V, E), \quad V = A \cup S \cup C \cup K,$$

where A is the set of artifacts, S the set of execution events, C the set of claims, and K the set of evidence contracts. Typed edges express relations such as *used*, *generated*, *supports*, *invalidates*, and *verified-by*. Iterative and feedback computations are represented by versioned events, making causal edges acyclic without claiming that the underlying scientific system or workflow specification is acyclic. This model can be mapped to the entities, activities, and agents of W3C PROV [1] and to the astronomy-specific IVOA Provenance Data Model [2].

For an artifact x , define the content identity

$$I_A(x) = H(C(x)),$$

where $\mathbf{C}$ is a versioned canonicalization function and $\mathbf{H}$ is a cryptographic hash. Canonicalization must specify encoding, field ordering, floating-point representation, missing values, endianness, and schema version.

For a step s with code p , environment e , parameters θ , and ordered input identities $\mathbf{i}$, define

$$I_S(s) = \mathbf{H}(I_A(p) \parallel I_A(e) \parallel \mathbf{C}(\theta) \parallel \mathbf{C}(\mathbf{i})).$$

The identity is a commitment to the declared execution context; it is not a claim that every property of a physical computer has been captured. Unobserved or unsupported dependencies must be represented explicitly as unknown.

4.1 Scientific acceptance

Bitwise equality is appropriate for some artifacts but too strict for many numerical and stochastic computations. Each claim therefore selects a directional verifier

$$V_K : (y_{\text{reference}}, y_{\text{candidate}}, \gamma) \longrightarrow \{\text{pass}, \text{fail}, \text{unknown}, \text{error}\},$$

where γ is the declared validation context. These rules are acceptance predicates, not generally equivalence relations. Examples include:

$$\begin{aligned} V_{\text{exact}}(y, y') &\equiv \mathbf{C}(y) = \mathbf{C}(y'), \\ V_{\text{tol}}(y, y') &\equiv d(y, y') \leq \epsilon, \\ V_{\text{unc}}(y, y') &\equiv (y - y')^\top \Sigma_\Delta^{-1} (y - y') \leq \tau, \end{aligned}$$

where d is a domain-appropriate distance, ϵ is a declared numerical tolerance, and Σ_Δ is a justified covariance model for the difference between the reference and candidate. If covariance, required inputs, or verifier preconditions are unavailable, the result is **unknown**, not **pass**. More specialized verifiers can test distributional agreement, conservation laws, physical units, coordinate consistency, monotonicity, model calibration, or domain quality flags.

A certificate is a tuple

$$\mathcal{E} = (G, B, Q, P, V, \Sigma),$$

where B is the declared observation boundary, Q the capture-coverage statement, P the replay procedure, V the validation policy, and Σ an optional collection of digital signatures or attestations. A release decision may be **pass** only when graph integrity holds, every mandatory atomic check passes, the policy permits no unresolved unknowns, and all required attestations validate.

5 Proposed User Interface

```
1 import origin
2
3 @origin.step(
4     name="calibrate_light_curve",
5     verifier=origin.allclose(rtol=1e-8, atol=1e-12),
6 )
7 def calibrate(raw_flux, flat_field):
8     return raw_flux / flat_field
9
10 with origin.run("transit-search") as run:
11     calibrated = calibrate(raw_flux, flat_field)
12     score = classifier.predict(calibrated)
13
14     candidate = run.claim(
15         "transit_candidate",
16         value=score,
17         uncertainty=score_uncertainty,
18         evidence=[calibrated, classifier],
19     )
20
21 candidate.verify()
22 candidate.export("ivoa-prov")
23 candidate.write("candidate.origin.json")
```

The primary interface is explicit. Convenience adapters may recognize NumPy arrays, Astropy quantities, FITS files, PyTorch models, or command-line programs, but adapter failure must be visible and must never silently downgrade the claimed completeness of a capsule.

6 Minimal Software Architecture

The core should remain small enough for complete review and independent reimplementaion:

```
origin/
|-- artifact.py      immutable, content-addressed artifacts
|-- step.py          execution records and decorators
|-- claim.py         claims and validation contracts
|-- graph.py         immutable evidence graph
|-- verify.py        verification and replay
|-- canonical.py     stable serialization and hashing
'-- adapters/
    |-- numpy.py
    |-- astropy.py
    |-- torch.py
    '-- ivoa.py
```

The preferred core has no mandatory dependency beyond the Python standard library. Domain

integrations are optional extras. Canonical JSON is the portable exchange representation; JSON Lines can support append-only event recording, and SQLite can provide a local indexed store. A separate, minimal `origin-verify` implementation should validate capsules without trusting the producer implementation.

7 Non-Negotiable Engineering Invariants

1. Published artifacts, nodes, and claims are immutable.
2. Equal canonical content has equal content identity.
3. Unsupported or unknown dependencies are reported explicitly.
4. The original exception type, value, and traceback are preserved.
5. Failed and interrupted executions are first-class records.
6. Historical metadata is versioned rather than mutated in place.
7. Capsule verification does not require executing untrusted code.
8. Replay is isolated from verification and requires explicit authorization.
9. Large artifacts are streamed and referenced rather than duplicated.
10. Sensitive metadata can be redacted before persistence without invalidating non-redacted integrity commitments.
11. Serialization is deterministic and governed by a public specification.
12. No hidden global state is required for correctness.

The system should offer graded evidence levels rather than a single ambiguous “reproducible” flag:

Grade	Name	Meaning
A	Exact	Independent replay is bitwise identical.
B	Numerical	Replay satisfies a declared numerical tolerance.
C	Scientific	Replay satisfies a domain-specific uncertainty or validity test.
D	Attributable	Dependencies and integrity are known, but replay is unavailable.
U	Incomplete	One or more required dependencies are unknown or unverifiable.

8 Applications in Space Science and AI

8.1 Astronomical data-product certification

For a derived image, spectrum, catalogue, or light curve, a capsule can record the observation identifiers, calibration products, masks, coordinate frames, units, software versions, parameters, uncertainty transformations, and quality tests. Export through the IVOA model makes the result interoperable with the astronomy community rather than creating an isolated provenance dialect.

8.2 AI decision evidence

For anomaly detection or mission decision support, a capsule can bind a telemetry interval to preprocessing code, model weights, training-data identity, calibration state, prediction, uncertainty, out-of-distribution score, operating envelope, and human review. The capsule certifies the evidence available to a decision; it does not claim that provenance alone makes an AI system safe.

8.3 Orbit determination and simulation

An orbital solution may identify observations, time standard, reference frame, ephemerides, force model, numerical integrator, solver settings, convergence criteria, covariance, and residual tests. A simulation capsule can similarly bind initial conditions, physical constants, discretization, random state, and conservation-law verifiers.

8.4 Calibration impact analysis

Content-addressed dependencies allow a changed calibration file, model, or algorithm to invalidate precisely the affected downstream claims. This supports selective recomputation and prevents stale products from silently remaining valid.

8.5 Candidate flagship study

A reproducible transit-search or variability-analysis pipeline using public TESS or Gaia products would provide a focused benchmark. Each processing stage would emit a capsule, an independent group would replay the analysis, and injected calibration, unit, model, and configuration faults would test detection and impact localization.

9 Verification and Testing Strategy

The redesigned concept is deliberately easy to test because it makes its support boundary explicit. Required test classes include:

- unit tests for every canonicalization and schema rule;
- property-based tests for graph and immutability invariants;
- golden test vectors shared with the independent verifier;
- malformed-input and adversarial-metadata fuzzing;
- mutation testing of the core implementation;
- cross-platform, endianness, and floating-point tests;
- crash-consistency and interrupted-execution tests;
- streamed hashing tests for very large artifacts;
- tampering, signature, and redaction tests;
- randomized graph and graph-migration tests;
- clean-environment replay by an independent team.

Initial engineering targets should include complete schema conformance, detection of every modification to committed content, no silent dependency loss in supported interfaces, bounded memory independent of artifact size, and measured overhead below five percent on representative file-level scientific pipelines. These are design targets, not established results, and must be evaluated empirically.

10 Research Questions and Experimental Design

The research paper should test falsifiable hypotheses rather than present only an architecture.

RQ1: Completeness.

How accurately does ORIGIN capture all dependencies declared by a supported scientific step?

RQ2: Reproduction.

How often can independent environments reproduce certified results at grades A, B, and C?

RQ3: Scientific equivalence.

Do uncertainty-aware contracts distinguish harmless numerical variation from scientifically meaningful errors better than exact hashes or fixed tolerances alone?

RQ4: Cost.

What execution-time, storage, memory, and developer-effort overhead does ORIGIN introduce?

RQ5: Diagnosis.

Does the evidence graph reduce the time required to locate injected data, code, calibration, configuration, or model faults?

RQ6: Interoperability.

Can capsules round-trip through W3C PROV and IVOA representations without losing required scientific semantics?

Baselines should include a plain script with a lockfile, a container-only workflow, noWorkflow [4], AiiDA [3], and an OpenLineage-compatible pipeline [5]. Experimental faults should alter individual data, calibration files, packages, model weights, seeds, units, coordinate frames, precision, preprocessing order, and configuration values. Outcomes should measure detection, localization, replay success, false alarms, and user effort.

11 Security, Privacy, and Assurance

Provenance may disclose filenames, queries, identities, model prompts, instrument states, or operational procedures. The threat model must cover malicious capsule modification, untrusted replay code, hash substitution, metadata injection, credential leakage, unauthorized graph access, and denial of service through large or cyclic inputs. Verification should be safe by default; replay should occur only inside an explicitly selected isolation boundary.

An evidence capsule is not by itself a flight-software qualification, safety case, or regulatory audit record. Nevertheless, the architecture supports repeatable testing and records of software, test environment, support tools, and discovered failures, which align with concerns in ECSS software product assurance [8]. Initial adoption should target ground-science and decision-support pipelines rather than safety-critical autonomous control.

12 Adversarial Evaluation of the Concept

This section deliberately adopts the position of a skeptical journal reviewer, mission-assurance engineer, security assessor, scientific user, and research software maintainer. The objective is not to weaken the project rhetorically, but to identify what must be true before its claims deserve trust. No finite analysis can enumerate every future defect or use case; the assessment below covers the principal mathematical, scientific, engineering, usability, security, adoption, and publication dimensions.

12.1 Summary judgment

The redesigned idea is substantially stronger than the universal-proxy prototype. It addresses a genuine problem, has a clear integrity-centered philosophy, and could become a useful interoperability and verification layer. However, the current proposal combines several ideas that already have mature precedents: research-object packaging, workflow-run provenance, content-addressed artifacts, signed supply-chain attestations, replayable environments, domain-specific reproducibility thresholds, and uncertainty-aware assessment. RO-Crate already packages research artifacts and metadata [9]; Workflow Run RO-Crate records interoperable workflow execution provenance [10]; ReproZip captures dependencies for replay [11]; and in-toto and SLSA define verifiable statements about how software artifacts were produced [12, 13]. Scientific-workflow research has also proposed domain-feature thresholds for automatic reproducibility assessment [14], validity constraints [15], and uncertainty-aware reproducibility for AI-enabled workflows [16].

Consequently, “evidence capsules” are not automatically a new scientific field or a novel method. Without a sharper theorem, semantics, algorithm, or empirical finding, reviewers may reasonably describe ORIGIN as a Python convenience layer or a synthesis of known techniques.

12.2 Complete inventory of strengths

Conceptual strengths

- It addresses the important distinction between recording history and demonstrating that a result remains valid.
- It replaces impossible universal interception with explicit scientific boundaries, making supported and unsupported behavior easier to define.
- It treats unknown dependencies as a result rather than silently pretending that capture was complete.
- It distinguishes integrity, attribution, replay, numerical reproducibility, and scientific validity instead of collapsing them into one word.
- It recognizes that bitwise equality is often inappropriate for numerical, stochastic, and hardware-dependent computation.
- It associates validation rules directly with the claims they justify.
- It can represent failed executions, warnings, and negative results as evidence rather than deleting them from the historical account.
- It separates verification from replay, reducing the need to execute code merely to inspect an artifact.

- It can support selective invalidation and recomputation when an upstream dependency changes.
- It encourages users to define the scientific meaning of successful reproduction rather than accepting a generic technical test.

Engineering strengths

- Immutable records are easier to reason about, cache, replicate, sign, and audit than mutable object histories.
- Content-derived identifiers facilitate deduplication, integrity checking, and comparison across independent executions.
- A small standard-library core could be reviewed more completely than a large workflow platform.
- Optional adapters allow scientific integrations without imposing all dependencies on every user.
- Canonical portable records can outlive the Python implementation.
- Streaming digests avoid copying large scientific products into a provenance database.
- A separately implemented verifier reduces common-mode defects between the producer and checker.
- Explicit schemas enable compatibility tests and independent implementations.
- Append-only event records support crash recovery and later indexing.
- Local JSON or SQLite operation permits use without a central service.
- Standards export could connect small Python projects to institutional archives and workflow engines.
- The decorator and context-manager API is familiar to Python researchers.
- The design can be introduced around only high-value products rather than requiring conversion of an entire codebase.

Scientific strengths

- Units, coordinate frames, uncertainty, calibration versions, and quality flags can be made explicit and automatically checked.
- A capsule can make a publication's computational evidence easier to inspect than an informal methods section and a repository link.
- Independent replay produces stronger evidence than successful execution by the original authors in the original environment.
- Fault-injection evaluation is falsifiable and can quantify detection and localization rather than relying on testimonials.
- The graph can support sensitivity and impact analysis across calibration and model changes.
- Domain-specific validators can test conservation laws, residual structure, physical constraints, catalogue invariants, and statistical calibration.
- The approach could expose undocumented assumptions that otherwise remain hidden until a workflow is moved to new data or infrastructure.
- It can preserve the relationship between an AI output, its model, its input interval, and its human review without claiming that the model is thereby correct.

Adoption strengths

- A lightweight library can serve teams that do not want a complete workflow management platform.

- Ground-science pipelines are a lower-risk and more attainable entry point than flight software.
- W3C PROV, IVOA, RO-Crate, and supply-chain attestation mappings could reduce institutional resistance if implemented as compatibility profiles.
- Human-readable summaries can provide immediate debugging value before an organization adopts advanced verification.
- A public conformance suite and independent verifier could make evaluation by agencies and journals relatively inexpensive.
- The core idea is domain-independent while adapters can remain domain-specific.

12.3 Mathematical and semantic weaknesses

Tolerance predicates must not be called equivalence relations

A naive formulation based on the predicate

$$d(y, y') \leq \epsilon$$

is generally reflexive and symmetric but not transitive. For example, three scalar results can satisfy $|a - b| \leq \epsilon$ and $|b - c| \leq \epsilon$ while $|a - c| > \epsilon$. Calling such a rule an equivalence relation is mathematically incorrect. The final design therefore treats it as a directional acceptance predicate relative to a fixed reference, version, and context. The same problem affects many relative-tolerance rules and can make grades change when the reference artifact changes.

The uncertainty rule

$$d(y, y') \leq k\sqrt{u_y^2 + u_{y'}^2}$$

implicitly treats the uncertainties as independent scalar standard deviations. Scientific results often share inputs and therefore have correlated errors. A more appropriate comparison may require a covariance matrix,

$$(y - y')^T \Sigma^{-1} (y - y') \leq \tau,$$

with a justified distribution, covariance estimate, degrees of freedom, and multiple-comparison policy. Even this does not establish scientific truth: an incorrect model can reproduce its own biased result with high precision.

Additional semantic problems include:

- The choice of distance, tolerance, uncertainty model, and threshold can be manipulated until a desired replay passes.
- A validator can be too weak: matching means, counts, or summary statistics may conceal scientifically decisive differences.
- A validator can be too strong and reject legitimate results produced by a better numerical method.

- Statistical tests have type-I and type-II errors and require correction when many artifacts or features are tested.
- Estimated uncertainty may itself be uncalibrated, conditional on a false model, or unavailable.
- Scientific validity frequently depends on external theory and observations, not merely agreement with a previous output.
- Reproducing an erroneous result is evidence of reproducibility, not correctness.
- A claim may be supported probabilistically, conditionally, or defeasibly; a simple graph edge cannot express all epistemic structure.

Identity is underspecified

Byte identity, semantic identity, and scientific identity are different. Two FITS files may encode scientifically equivalent arrays with different header ordering, compression, timestamps, or padding. Conversely, two byte-identical files may be interpreted differently under different schemas, units, calibration knowledge, or external reference data.

Canonicalization is therefore a major research and maintenance problem rather than a minor utility function. It must address NaNs with payloads, signed zero, endianness, Unicode normalization, dictionary ordering, timestamps, archive metadata, object graphs, sparse formats, compressed data, and format evolution. Domain-aware canonicalization risks changing scientific meaning; byte-level canonicalization misses semantic equivalence.

Step identity is also problematic. A source-code hash does not capture dynamically loaded libraries, native code, compiler behavior, environment variables, external services, filesystem state, database snapshots, kernel behavior, firmware, microcode, scheduler nondeterminism, or human actions. Capturing all of them is usually infeasible, while omitting them weakens the meaning of the commitment.

A DAG is useful but not universally natural

Iterative solvers, feedback systems, adaptive experiments, online learning, and cyclic workflow specifications are not naturally DAGs. They can be represented as an acyclic sequence of versioned events, but that transformation must be specified. Graphs may also become extremely large, and a coarse step-level graph may falsely imply that every output depends on every input. Fine-grained dependency annotations reduce false positives but increase authoring burden and create another source of error.

12.4 Scientific weaknesses

- Provenance cannot compensate for invalid experimental design, biased data, data leakage, inappropriate statistics, or a false physical model.
- A capsule may create unjustified confidence through polished metadata and cryptographic terminology.
- Scientific claims evolve when new calibrations, theories, catalogues, or quality criteria appear; validity is not permanently determined at creation.

- Reproduction may be impossible because data are proprietary, personal, export-controlled, embargoed, deleted, or too large to redistribute.
- Long-term replay may fail because hardware, licenses, services, and package repositories disappear.
- Hardware-sensitive simulations can require facilities unavailable to an independent reviewer.
- Stochastic AI training can be prohibitively expensive to repeat, so the capsule may verify identity without verifying the training claim.
- Model weights and output scores do not reveal whether training data were lawfully obtained, representative, or free of contamination.
- Out-of-distribution and uncertainty scores are method-dependent and can be unreliable outside their validation regime.
- A single astronomy demonstrator would not establish usefulness in mission operations, AI assurance, planetary science, or other fields.
- A tool built by its evaluators is susceptible to benchmark selection, confirmation bias, and favorable definitions of success.

12.5 Engineering weaknesses

The “small library” and “complete evidence” goals conflict

A genuinely small core can hash files and serialize records, but robust evidence capture across NumPy, Astropy, FITS, PyTorch, containers, databases, HPC schedulers, remote object stores, notebooks, compilers, and mission archives requires a large and continuously maintained integration surface. Optional adapters move complexity; they do not remove it.

Other engineering risks include:

- Hashing multi-terabyte products adds I/O cost even when memory is bounded.
- Remote resources can change after a URI is recorded; strong identity then requires versioned immutable storage or a trusted archive digest.
- Canonical schemas require migration, backward compatibility, and hash algorithm agility over decades.
- Content hashes disclose equality and can support dictionary attacks against low-entropy sensitive data.
- Digital signatures require key generation, custody, rotation, revocation, time stamping, and organizational trust policies.
- A signature proves possession of a key, not that the recorded experiment was honest or uncompromised.
- A compromised producer can faithfully sign false metadata.
- Safe verification still parses attacker-controlled structures and must resist resource exhaustion, path traversal, decompression bombs, malicious links, and parser differentials.
- Replay executes untrusted scientific code and therefore needs isolation, resource limits, network policy, and secret management outside the scope of a small Python package.
- Redaction can break dependency interpretation. Cryptographic commitments to redacted values preserve integrity but may leak information or prevent independent scientific assessment.
- SQLite and JSON are convenient locally but do not by themselves solve distributed concurrency, access control, retention, discovery, or institutional preservation.
- The independent verifier can drift from the producer specification or share ambiguities even if it

does not share code.

- Python is appropriate for scientific tooling but may not satisfy the coding, qualification, determinism, or long-term maintenance constraints of every operational environment.

Explicit capture shifts error to the user

Explicit boundaries avoid magical interception but create declaration risk. Users can forget an input, mislabel an artifact, choose an inadequate verifier, omit a random source, or record an environment that is only approximately correct. The system may then produce a cryptographically intact certificate of an incomplete or false statement. Completeness cannot be inferred merely because a decorator ran.

Automatic capture and explicit declaration therefore have complementary failure modes: automation misses behavior it cannot observe, while declarations omit what authors do not know or choose not to report. A strong design needs measured coverage, negative evidence, and mechanisms for independent challenge.

12.6 Usability and incentive weaknesses

- Researchers often perceive provenance work as overhead that benefits future users more than the person doing the work.
- Mandatory declarations can make concise exploratory code verbose.
- Users may select “exact” or “allclose” mechanically without understanding the associated scientific claim.
- Evidence graphs and signatures introduce concepts unfamiliar to many domain scientists.
- Producing a capsule is easier than preserving every referenced artifact and maintaining replay infrastructure.
- Teams already using AiiDA, Nextflow, Snakemake, CWL, MLflow, institutional archives, or data-management systems may see little incremental benefit.
- If the tool requires adapters and policy files before showing value, users may abandon it before reaching the verification benefits.
- If it records too much, it creates noise, storage cost, and privacy risk; if it records too little, it fails its central purpose.
- Human-readable summaries can become misleading simplifications of complex evidence and trust relationships.
- Researchers may resist a tool that exposes irreproducibility without providing resources to correct it.

12.7 Why the library might not be useful

The library may add another metadata format without changing scientific practice. It may tell users that a file changed when Git, an archive identifier, or an existing checksum already provides that information. It may duplicate workflow engines for structured pipelines while remaining less capable than system-call capture tools for unmodified legacy programs. If capsules do not integrate with publication systems, archives, CI services, workflow engines, and institutional identity systems, they may become sidecar files that nobody reads.

The primary bottleneck in reproducibility is often not absence of a Python API but missing data access, obsolete dependencies, inadequate documentation, insufficient compute, weak incentives, or invalid methods. ORIGIN cannot independently solve these problems. A perfectly recorded but unavailable environment is not reproducible; an executable but scientifically invalid pipeline is not trustworthy.

It may also be too narrow for institutions and too elaborate for individuals. Institutions need governance, catalogues, policy enforcement, distributed storage, identity, and decades-long stewardship. Individual researchers may prefer a repository, lockfile, container, and short README. The project must demonstrate a valuable middle ground rather than assume one exists.

12.8 Why the library might be very useful

The strongest use case is not universal provenance. It is a lightweight, independently verifiable bridge between an ordinary scientific Python function and established archival or workflow standards. It could be valuable when:

- a small research team lacks a workflow engine but produces important derived products;
- a reviewer needs a compact integrity and replay manifest;
- calibration changes require precise downstream impact analysis;
- a CI system must verify that a scientific result remains within a declared physical or statistical tolerance;
- multiple workflow systems need a common verification profile;
- an AI-assisted decision needs a durable link to model, input, preprocessing, uncertainty, and human disposition;
- an archive wants richer processing evidence without requiring all producers to adopt one execution platform;
- a team needs explicit grades that distinguish attribution from numerical or scientific reproduction;
- tampering detection and reproducibility assessment must be combined in one portable record.

The project could therefore succeed as a sharply scoped protocol implementation, conformance suite, and verifier even if it never becomes a universal execution tracker.

12.9 Why it may not be a genuine scientific contribution

A scientific contribution normally establishes new knowledge, a new method with demonstrated advantages, a formal result, or a consequential empirical finding. The following are engineering outputs but are not necessarily new science:

- defining Python dataclasses for artifacts, steps, and claims;
- hashing content and code;
- serializing a DAG to JSON;
- adding decorators to functions;
- exporting W3C PROV or IVOA records;
- signing metadata;
- replaying a container;
- comparing arrays with tolerances;

- packaging evidence and dependencies together.

Each is useful, but close relatives already exist. The proposed graded evidence scale is also vulnerable to the criticism that it is a relabeling of established levels of reproducibility. Domain-threshold validation has already been implemented for workflow outputs [14], and uncertainty-aware reproducibility has already been proposed for scientific ML workflows [16]. A paper must therefore state precisely what is new relative to these works.

Potentially genuine contributions would require substantially more depth, such as:

1. a formal semantics for evidence completeness with soundness and failure guarantees;
2. a validator algebra that distinguishes non-transitive acceptance predicates from equivalence and correctly composes uncertainty with covariance;
3. a protocol for cryptographically verifiable redaction that preserves useful graph reasoning;
4. an algorithm for minimal evidence extraction with proven dependency coverage under stated assumptions;
5. an empirical discovery showing that executable evidence contracts detect important scientific errors missed by existing systems;
6. a standardized interoperability profile accepted and implemented by multiple independent workflow systems or archives.

Even these must be supported by proofs, experiments, independent implementation, or real adoption rather than asserted as design intentions.

12.10 Why journals might reject the paper

Nature Computational Science asks reviewers to assess validity, originality, significance, methodological transparency, uncertainty treatment, and whether conclusions are robust and reproducible [17]. Under those criteria, likely rejection reasons include:

- **Insufficient novelty.** The work may appear to combine RO-Crate, workflow provenance, ReproZip, in-toto/SLSA, and validation thresholds.
- **No research result.** A proposal, architecture, and roadmap are not empirical or theoretical findings.
- **Overclaiming.** References to NASA, ESA, trustworthy AI, and scientific validity may exceed the evidence.
- **Weak formalism.** Hash definitions and a tolerance inequality are not by themselves a formal theory; the current acceptance predicates have unresolved mathematical problems.
- **Inadequate baselines.** Comparing only with the obsolete proxy or a lockfile would ignore mature provenance and reproducibility systems.
- **Toy evaluation.** One small transit-search notebook would not show generality, scalability, or mission relevance.
- **Self-defined success.** Authors could choose convenient validators, faults, workloads, and thresholds that favor their own system.
- **No independent replication.** Claims about independent verification are weak if all experiments are performed by the tool’s developers.
- **No user evidence.** There is no evidence that scientists understand, want, or correctly use the evidence contracts.

- **No agency evidence.** A hypothetical NASA/ESA application is not an agency requirement, collaboration, endorsement, or deployment.
- **No scientific discovery.** An astronomy journal may judge the tool secondary unless it enables a substantive astronomical result.
- **Scope mismatch.** A software package may fit a research-software venue better than a broad high-impact science journal.
- **Incomplete sustainability.** Long-term governance, schema evolution, archive preservation, security response, and maintenance are unresolved.
- **Unavailable implementation.** The redesigned library, verifier, standard, datasets, and evaluation do not yet exist.

Even the Journal of Open Source Software requires useful research software and substantial scholarly effort; a small wrapper around existing standards can be rejected as insufficiently substantial or insufficiently impactful [18]. Publication becomes credible only after the software is functional, documented, tested, used in research, and clearly differentiated.

12.11 Why NASA may not need it

NASA is not starting from an absence of identifiers, metadata, processing history, or provenance systems. The Planetary Data System has the PDS4 information model, stable product identifiers, discipline dictionaries, and a Processing Information dictionary describing processing history [19, 20]. NASA Earthdata systems already manage metadata and processing lineage, and NASA-supported work has applied provenance tools to standard data products [21, 22].

NASA programs may therefore reject ORIGIN because:

- it duplicates existing archive metadata, identifiers, workflow logs, or mission-specific processing records;
- it introduces a new schema and long-term maintenance obligation;
- its Python API may not integrate with legacy, distributed, or non-Python production systems;
- agency users may prefer extensions to PDS4, Earthdata standards, RO-Crate, or an existing workflow engine;
- content hashing and environment capture may be costly at mission scale;
- sensitive, proprietary, export-controlled, or security-relevant metadata may not be suitable for portable capsules;
- a third-party project without funded maintenance, governance, support, training, or security response creates operational risk;
- mission software adoption requires requirements traceability, validation, configuration control, documentation, and assurance evidence far beyond ordinary open-source tests;
- scientists may not perceive enough benefit over established practices to justify integration effort;
- a 2027 schedule provides little time for procurement, security review, pilot evaluation, and institutional adoption.

NASA might be interested only if ORIGIN fills a demonstrated gap between existing systems, exports directly into required agency representations, and is validated on a real NASA-affiliated pipeline with an internal collaborator.

12.12 Why ESA may not need it

ESA projects operate within ECSS engineering and software-product-assurance processes. ECSS-Q-ST-80 requires controlled procedures, repeatable tests, recorded software and test environments, problem reporting, configuration identification, and, where justified, independent verification and validation [8]. An evidence capsule can support pieces of this process but cannot replace it.

ESA or its contractors may decline the library because:

- project assurance is requirements-driven and contractually controlled, not satisfied by adopting a generic provenance package;
- existing mission consortia have established data models, archives, workflow systems, configuration management, and validation procedures;
- a new tool must itself be qualified, configured, documented, maintained, and placed under change control;
- the library does not generate requirements traceability, safety analyses, test procedures, nonconformance records, or acceptance evidence by itself;
- scientific-equivalence validators may be unacceptable without mission- and instrument-specific justification;
- agency and industrial partners may require languages, platforms, support contracts, or cybersecurity controls outside the proposed scope;
- interoperability with IVOA is relevant to astronomy archives but not sufficient for all ESA engineering and Earth-observation contexts;
- the operational cost of preserving referenced environments and artifacts may dominate the cost of the library;
- a small external team may not offer the continuity expected over a multi-decade mission lifecycle.

The credible objective is therefore not “ESA needs a new provenance system.” It is “a specific ESA-affiliated team has an unmet verification problem that this standards-compatible component solves better than its alternatives.”

13 Adversarial Evaluation of the Existing Code

The uploaded implementation is the original proxy prototype, not an implementation of the evidence-capsule design. Its modular organization and concise API are useful starting points, but it is not safe to reuse as the redesigned core.

13.1 Concrete strengths of the prototype

- The code is short and readable enough for rapid experimentation.
- Nodes, dispatch, proxy behavior, NumPy adaptation, and public API are separated into distinct modules.
- Parent tuples and UUID identity provide a simple in-memory DAG structure.
- `__slots__` reduces some per-object overhead.
- Delayed imports avoid immediate circular-import failures.
- Basic scalar arithmetic and basic top-level NumPy operations can produce readable histories.

- Reflected arithmetic operators and official NumPy dispatch protocols are at least considered.
- The implementation is small enough to replace rather than being trapped by a large incompatible architecture.

13.2 Demonstrated correctness failures

Behavioral probes of the uploaded modules showed the following:

- `wrap(5) * 2` records the multiplication and wrapped source but not the literal operand 2, contradicting the documented trace.
- `wrap("hello").upper()` produces a method node with no source parent; ordinary method calls sever lineage because the bound receiver is omitted from dispatch arguments.
- `list.append()` returns the proxy instead of Python's normal `None`, changing program semantics.
- Appending or assigning a wrapped value stores the wrapper inside the raw container and fails to record the inserted value as a dependency.
- Iteration returns raw elements, so element-level provenance disappears.
- Nested arguments are not recursively unwrapped despite the dispatch documentation. For example, joining a list of wrapped strings fails.
- Exceptions are reconstructed from a new message, altering their arguments and potentially masking the original exception when its constructor does not accept one string.
- `numpy.mean` works in a basic probe, while ufunc reductions and nested operations such as concatenating a list of wrapped arrays fail.
- `numpy.asarray` silently strips the wrapper and all tracking.

13.3 Structural defects in the prototype

- The argument collector sees only top-level wrapped values and ignores wrappers inside lists, tuples, dictionaries, dataclasses, and object fields.
- Non-callable attributes, iteration, length, membership, conversions, and truth testing bypass provenance.
- Mutation detection is a hard-coded set of method names. It both misses real mutators and misclassifies unrelated custom methods with the same names.
- Mutating calls omit argument parents, do not consistently unwrap arguments, and return raw outputs for methods such as `pop`.
- Item assignment records neither wrapped keys nor wrapped values as parents.
- Direct access to `_value` and public mutation of node fields allow tracking to be bypassed or historical records to be rewritten.
- Node identifiers, operations, parents, and metadata are mutable. Changing an identifier after hashing the node can corrupt sets and dictionaries.
- Existing wrapped results can have their provenance node overwritten, destroying prior history.
- Metadata dictionaries are shared and returned directly, enabling unrecorded mutation of historical nodes.
- Graph rendering uses breadth-first traversal with tree-style indentation, silently suppresses repeated shared nodes, and provides no cross-reference.
- The NumPy adapter handles only `--call--` ufuncs, shallowly unwraps values, wraps tuples as single values, and catches broad exceptions that hide the real error behind `NotImplemented`.

- In-place operators, matrix multiplication, shifts, calls, contexts, async behavior, descriptors, copying, pickling, and many other Python protocols are missing.
- The type annotations rely heavily on `Any`, bare `Origin` returns, and suppressions. They do not justify the README’s full type-safety claim.
- The uploaded package initializer is for the adapters subpackage rather than the documented public root API; as supplied, the file layout does not support `from origin import wrap, trace`.

The decisive defect is epistemic rather than cosmetic: the library can emit a convincing lineage graph that omits actual dependencies. A provenance system that silently produces incomplete histories must not describe those histories as complete.

13.4 Code-level conclusion

The proxy prototype should be retained only as a negative baseline and a source of test cases. The redesigned implementation should begin from immutable artifact, step, claim, policy, and capsule records. Attempting to incrementally repair the proxy would consume substantial effort while preserving the wrong abstraction and an unattainable transparency promise.

14 Revised Research Position

The defensible project is narrower than the original proposal:

ORIGIN should be an independently verifiable, astronomy-oriented compatibility profile that combines Workflow Run RO-Crate provenance, supply-chain attestations, and rigorously specified scientific acceptance predicates, with a minimal Python producer and a separately implemented verifier.

This positioning avoids inventing another general provenance vocabulary. The research contribution must focus on one unresolved question, preferably the sound composition of scientific acceptance predicates and dependency evidence under uncertainty. The first implementation should support one complete use case—for example, calibration and candidate validation in an astronomical light-curve pipeline—and should interoperate with existing standards from the beginning.

Proceed only if a systematic literature review and interviews with real users confirm a gap not already met by Workflow Run RO-Crate, ReproZip, AiiDA, in-toto, SLSA, and domain archives. If that gap cannot be demonstrated, the scientifically honest decision is to contribute the needed feature to an established project rather than create a new library.

15 Final Redesign: ORIGIN Gate

15.1 The premise that cannot be engineered away

No scientific software can be made “100% excellent,” eliminate every weakness, or guarantee adoption by NASA or ESA. It is impossible to guarantee complete observation of an open computational

system, permanent reproducibility of external resources, correctness of scientific assumptions, absence of implementation defects, or institutional procurement decisions. A masterpiece is not a system that denies these limits. It is a system that makes its guarantee boundary precise, fails safely outside that boundary, and supplies enough evidence for independent challenge.

The project should therefore replace absolute goals with four auditable promises:

1. **Integrity:** modification of committed content is detected under stated cryptographic assumptions.
2. **No silent completeness claim:** missing mandatory evidence produces `unknown`, never `pass`.
3. **Policy soundness:** the reference gate returns `pass` only if every mandatory check required by the selected policy has passed.
4. **Interoperability:** records conform to existing standards and can be verified without trusting the producing Python process.

These promises are strong, useful, testable, and honest. They do not assert that a passing product is scientifically true, safe for flight, unbiased, or complete outside its declared observation boundary.

15.2 One-sentence product definition

ORIGIN Gate is a CI-ready release gate that evaluates machine-readable evidence contracts for scientific data products and AI-assisted results, emits interoperable attestations, and refuses to pass when required evidence is absent.

This is not a workflow engine, package manager, container runtime, archive, experiment tracker, universal Python proxy, or new general provenance language. It is the thin assurance layer between those systems and a consequential product release or scientific claim.

15.3 The concrete real-world problem

Scientific teams routinely ask whether a derived product is ready to release after a change to data, calibration, code, model weights, environment, or infrastructure. Current answers are often distributed across checksums, test logs, workflow metadata, notebooks, quality-control scripts, and expert memory. ORIGIN Gate turns the release decision into a versioned contract with a portable result.

For an astronomical data product, a contract may require:

- byte integrity and a recognized FITS schema;
- declared source observations and calibration identifiers;
- valid units, coordinate frames, time standards, and array shapes;
- finite-value, mask, saturation, and quality-flag constraints;
- astrometric, photometric, or residual tolerances justified by the instrument;
- a recorded workflow run and immutable software-environment reference;
- successful comparison with a reference product under a covariance-aware acceptance policy;
- required review or signature attestations.

For an AI-assisted ground-operations product, a contract may require a model digest, preprocessing identity, evaluation-set reference, operating-domain declaration, calibration metrics, uncertainty

output, out-of-distribution policy, and human disposition. The gate verifies that this evidence exists and satisfies policy; it does not certify that arbitrary AI behavior is safe.

15.4 Five user-visible objects

The final design needs only five principal objects:

Artifact manifest.

Byte digest, media type, size, stable external identifiers, optional semantic profile, and explicitly versioned metadata.

Observation boundary.

The processes and dependency classes the producer claims to observe, plus explicit exclusions such as external services, hardware nondeterminism, or undocumented manual actions.

Evidence contract.

A versioned list of mandatory and advisory checks, their preconditions, reference artifacts, parameters, and composition policy.

Verification report.

Atomic results, measurements, reasons, tool identities, and the aggregate outcome **pass**, **fail**, **unknown**, or **error**.

Attestation bundle.

The workflow-run record, contract, report, artifact commitments, and optional signatures packaged using established formats.

15.5 Command-line experience

The first release should optimize for a five-minute successful evaluation:

```
origin init astronomy-fits
origin check product.fits --contract release.yaml
origin attest product.fits --report origin-report.json
origin verify origin-bundle.crate
origin diff previous.crate current.crate
origin impact calibration-v4.fits --index release-index.json
```

The default command must produce a concise explanation, not merely a status code:

```
OUTCOME: UNKNOWN (release blocked)
PASS     byte integrity, FITS schema, units, WCS, finite-value policy
FAIL     photometric residual p95 = 0.0174 > 0.0100
UNKNOWN  calibration artifact cal-438 has no immutable digest
INFO     workflow run and OCI environment are attached
```

The distinction among failure, missing evidence, and verifier malfunction is essential. A binary success flag would recreate the ambiguity the project is meant to remove.

15.6 Python interface

```
1 from origin import Contract, artifact, gate
2 from origin.profiles import fits, numpy_validation
3
4 release = Contract("tess-light-curve-release", version="1.0")
5 release.require(fits.valid_schema("light-curve-v2"))
6 release.require(fits.units(flux="electron / s", time="d"))
7 release.require(numpy_validation.finite(fraction=0.999))
8 release.require(
9     numpy_validation.mahalanobis(
10         reference="baseline.fits",
11         covariance="calibration-covariance.npz",
12         threshold=16.3,
13     )
14 )
15 release.require_evidence("workflow-run-ro-crate")
16 release.require_evidence("immutable-environment")
17
18 report = gate.evaluate(
19     artifact("candidate.fits"),
20     contract=release,
21     boundary="ground-pipeline-v1",
22 )
23
24 report.require_pass() # raises on FAIL, UNKNOWN, or ERROR
25 report.write("origin-report.json")
```

Validators should be pure, deterministic functions over explicit inputs wherever possible. Plugins that invoke external programs must run in a controlled subprocess with explicit timeout, resource, filesystem, and network policies.

15.7 Standards-first architecture

ORIGIN Gate should own as little vocabulary and infrastructure as possible:

Concern	Recommended foundation
Research-object packaging	RO-Crate and Workflow Run RO-Crate
General provenance semantics	W3C PROV
Astronomy mapping	IVOA Provenance Data Model
Attestation envelope	in-toto statement and DSSE-compatible envelope
Supply-chain records	SLSA-compatible subjects and materials
Runtime preservation	OCI digest, lockfile, Nix/Guix reference, or institutional environment identifier; ORIGIN does not build containers
Signatures and identity	Established signing services such as Sigstore or institutional PKI; no custom cryptography
Large data identity	Archive checksum and persistent identifier, optionally revalidated by streaming digest
Contract syntax	A small versioned JSON Schema with a deterministic canonical representation

Workflow Run RO-Crate is the container for execution evidence, not merely an export target. ORIGIN adds a narrow contract and verification profile. This reduces schema proliferation and makes it possible for other languages and workflow systems to produce valid bundles without importing Python.

NASA’s current science strategy calls for transparent, trustworthy, reproducible open science and open-source tools [6]. ESA’s EarthCODE and Open Science Catalog already support publication of reproducible workflows [25]. These programs are both opportunity and warning: ORIGIN must integrate with existing ecosystems rather than present itself as their replacement.

15.8 Byte identity and semantic fingerprints

Every artifact has one mandatory byte digest. Format-aware semantic fingerprints are optional profile outputs and never replace the byte digest. This resolves a major ambiguity:

- the byte digest answers whether the artifact changed;
- the semantic profile answers whether selected scientific content is equivalent under a versioned interpretation;
- the acceptance contract answers whether the change is permitted for this release decision.

For example, a FITS profile may normalize irrelevant header order for comparison, but the original file’s byte digest remains the integrity identity. Profile output must include its profile name and version so that future software can reproduce the interpretation.

15.9 Contract outcome logic

Let a contract K contain mandatory atomic checks $a_1, \dots, a_n$ and a policy π . Each atomic check returns a structured result

$$r_i = (s_i, m_i, q_i), \quad s_i \in \{\text{pass}, \text{fail}, \text{unknown}, \text{error}\},$$

where m_i contains measurements and q_i contains diagnostic evidence. The strict default conjunction is

$$\text{Gate}(K) = \begin{cases} \text{fail}, & \exists i : s_i = \text{fail}, \\ \text{error}, & \nexists i : s_i = \text{fail} \wedge \exists i : s_i = \text{error}, \\ \text{unknown}, & \forall i : s_i \notin \{\text{fail}, \text{error}\} \wedge \exists i : s_i = \text{unknown}, \\ \text{pass}, & \forall i : s_i = \text{pass}. \end{cases}$$

Alternative policies may permit explicitly named advisory unknowns, but they must never silently weaken mandatory checks. A policy is itself content-addressed and included in the attestation.

The core soundness property to prove and test is:

If the gate returns **pass**, then every mandatory atomic check in the exact committed contract returned **pass**, every required evidence object was present and integrity-valid, and the verification report was deterministically derived from those results by the committed policy.

This is a tractable software property. It deliberately does not claim that every atomic validator is scientifically sufficient; that obligation belongs to the profile specification and its domain validation.

15.10 Capture coverage rather than absolute completeness

Every bundle should include a machine-readable coverage matrix:

Dependency class	Status	Evidence
Declared input files	observed	byte digests and archive identifiers
Python packages	observed	lockfile and installed-distribution digests
Native libraries	partial	loader inventory; transitive system state unknown
Randomness	observed	generator algorithm and initial state
External HTTP services	excluded	network disabled during certified run
Manual actions	unknown	no authenticated event record
Hardware nondeterminism	partial	device and driver recorded, microstate unknown

Completeness claims are then conditional: “complete for mandatory file, package, randomness, and network dependencies within boundary B ,” not “complete history of the computation.” A required class marked partial, excluded, or unknown blocks the gate unless the policy explicitly justifies it.

15.11 Minimal implementation

The version-1 core should contain approximately the following responsibilities:

```
origin/
|-- model.py          immutable contract and report models
|-- canonical.py      deterministic JSON and digest rules
|-- evaluate.py       four-outcome policy engine
|-- bundle.py         Workflow Run R0-Crate integration
|-- verify.py         safe integrity and policy verification
|-- cli.py            check, attest, verify, diff, impact
|-- profiles/
|   |-- generic.py
|   '-- astronomy_fits.py
'-- plugins.py        isolated validator protocol
```

The first release should not include transparent proxies, automatic Python operation tracing, a database server, a web application, container construction, custom cryptography, a generic workflow scheduler, PyTorch training capture, or every proposed domain adapter. Each would expand the attack surface and delay the one complete use case.

15.12 Engineering-masterpiece criteria

The project should be judged by discipline rather than feature count:

- a public normative specification precedes implementation growth;
- immutable typed models reject unknown or ambiguous fields by default;
- every serialized field has defined semantics, units, optionality, and version behavior;
- parsing, integrity verification, policy evaluation, and replay are separate trust domains;
- verification never imports plugins or executes producer code;
- all network access is opt-in and visible in the report;
- deterministic output is tested across supported Python versions and operating systems;
- the CLI has stable exit codes for pass, fail, unknown, error, and malformed input;
- errors preserve cause, location, and structured diagnostics;
- memory use is bounded for large files through streaming;
- schemas and hash algorithms have explicit migration and deprecation plans;
- security policy, vulnerability reporting, signed releases, SBOMs, and reproducible package builds exist before version 1.0;
- API stability is protected by compatibility tests and semantic versioning;
- the specification and conformance suite are usable by non-Python implementations;
- documentation contains failure cases and threat boundaries, not only happy examples.

15.13 Easy and unusually strong testing

Testing should be a principal product, not a support activity. The repository should publish a conformance corpus containing valid bundles and one-fault mutants. Each fixture states its required

result and explanation.

1. **Canonicalization vectors:** Unicode, numeric boundaries, NaNs, signed zero, ordering, large integers, invalid encodings, and schema migrations.
2. **Policy truth tables:** exhaustive combinations of pass, fail, unknown, and error for every composition operator.
3. **Tampering corpus:** modified artifact, contract, report, signature, workflow record, and dependency edge.
4. **Astronomy fault corpus:** wrong unit, frame, time scale, calibration, mask, shape, WCS, quality flag, covariance, and reference product.
5. **Resource attacks:** oversized fields, deep structures, duplicate identifiers, link cycles, path traversal, archive bombs, and timeout cases.
6. **Property tests:** serialization idempotence, digest stability, immutability, graph closure, and monotonic blocking when mandatory evidence is removed.
7. **Differential verification:** the Python producer and an independent verifier must agree on every golden vector.
8. **Mutation testing:** changes to policy precedence, digest coverage, or schema validation must be detected by the suite.
9. **Performance tests:** streaming throughput, peak memory, startup time, report size, and large-graph traversal.
10. **Independent replay:** a second institution runs the complete public benchmark without assistance from the developers.

The core gate can be exhaustively tested more easily than a universal provenance proxy because its inputs are finite documents and deterministic validators.

15.14 Flagship astrophysics demonstrator

The initial demonstrator should be a release gate for a public astronomical light-curve product. It should use a synthetic corpus with known faults and a real public dataset. The workflow should include raw-product selection, calibration, detrending, candidate scoring, uncertainty propagation, and final product release.

The study should compare four conditions:

1. repository, README, and lockfile only;
2. container and checksums;
3. established workflow provenance without ORIGIN contracts;
4. the same workflow plus ORIGIN Gate.

Inject faults into data, calibration, units, coordinate systems, code, environment, random state, model weights, covariance, and policy. Measure detection recall, false blocking, localization time, runtime overhead, report size, contract-authoring effort, and independent replay success. A blinded evaluator should select or inject some faults so the developers cannot tailor the validators to every case.

15.15 The genuine research contribution

The paper should not claim invention of provenance, research objects, attestations, or reproducibility levels. Its central claim should be narrower:

A four-outcome, boundary-relative evidence-contract gate can convert heterogeneous workflow provenance and scientific validation results into a fail-safe release decision, with independently verifiable policy soundness and measurably improved fault detection for scientific data products.

The research contributions would be:

1. formal semantics for boundary-relative evidence coverage and four-outcome scientific release policies;
2. a standards-compatible contract profile rather than a new provenance model;
3. a covariance-aware astronomy validation profile;
4. an adversarial scientific-product fault corpus and conformance suite;
5. an independent implementation and empirical comparison with established approaches;
6. evidence about user correctness, overhead, and fault-localization benefit.

The strongest theorem is modest but meaningful: under the specified verifier and cryptographic assumptions, a **pass** report implies successful execution of all committed mandatory checks over integrity-valid committed inputs. Scientific adequacy remains an empirical property of the domain profile.

15.16 Global project strategy

Global recognition requires an ecosystem strategy in addition to code:

- use a permissive license with clear patent terms and contribution policy;
- publish the specification under an open documentation license;
- establish a lightweight technical steering group with astronomy, statistics, security, workflow, and archive representation;
- require public design proposals for schema or policy changes;
- maintain a compatibility matrix and long-term-support release;
- create a public conformance badge based on test results, not membership;
- provide examples for local laptops, HPC systems, and CI services;
- make documentation accessible and invite translation only after the core terminology stabilizes;
- publish negative results, rejected designs, benchmarks, and security incidents transparently;
- avoid branding that implies NASA or ESA endorsement without written authorization.

The project needs at minimum a research software engineer, a domain astronomer, a statistician or uncertainty-quantification expert, a security reviewer, and an external pilot user. A single developer can build the prototype, but cannot provide credible independent validation, domain authority, security assessment, community governance, and institutional adoption alone.

15.17 A credible agency-adoption funnel

Technical quality cannot make NASA or ESA adoption “very likely” by itself. Adoption depends on an identified owner, budget, requirements, integration cost, security review, maintenance, and trust. The 2027 goal must be defined precisely:

Attainable 2027 objective.

One NASA-, ESA-, observatory-, or mission- affiliated research group uses ORIGIN Gate in a non-flight public pilot or CI release check.

Stretch 2027 objective.

The group publishes an independent evaluation or integrates the gate into a recurring ground-science workflow.

Not credible for 2027.

Agency-wide standardization, operational flight use, safety certification, or replacement of institutional provenance infrastructure.

The fastest route is to solve one partner’s existing release problem and export into its required metadata. NASA open-science programs fund open tools and require stronger data and software management in funded research [23, 24]. ESA Earth-observation programs already emphasize FAIR workflows, interoperability, and reproducibility [25]. These are possible entry points, not evidence that either agency needs this particular library.

15.18 Date-specific execution plan

Because the current date is August 19, 2026, a 2027 external pilot requires an aggressively narrow schedule:

Deadline	Required evidence of progress
Sep. 15, 2026	Complete a systematic competitor matrix, twenty structured user interviews, one astronomy product specification, and a written decision on whether to extend Workflow Run RO-Crate instead of launching a separate standard.
Oct. 15, 2026	Publish contract schema v0.1, threat model, outcome semantics, twenty golden vectors, and a command-line prototype supporting generic files and one FITS profile.
Nov. 15, 2026	Obtain a named external co-design partner, freeze the pilot requirements, and demonstrate fault detection on a synthetic light-curve corpus.
Dec. 15, 2026	Release alpha software, independent verifier, public benchmark, signed releases, security policy, documentation, and CI integrations.
Mar. 31, 2027	Complete the first external pilot and publish all non-sensitive reports, failures, overhead measurements, and contract-authoring effort.
Jun. 30, 2027	Complete independent replication, user study, comparison baselines, and a paper-quality empirical evaluation.
Sep. 30, 2027	Submit the software-engineering paper and release version 1.0 only if the conformance, security, and usability thresholds are met.
Dec. 31, 2027	Secure a second independent integration or stop claiming broad agency relevance and reposition the project as a focused research tool.

15.19 Quantitative release and research thresholds

Proposed targets must be preregistered before the final benchmark:

- all normative conformance vectors produce identical results in two independent verifier implementations;
- every committed-content mutation in the tampering corpus is detected;
- removing mandatory evidence never changes **pass** to another **pass**;
- at least 95% of seeded in-scope scientific-product faults are detected, with every miss classified and published;
- false blocking on accepted reference products remains below a preregistered domain threshold;
- peak memory remains bounded during streaming validation of large files;
- median overhead remains below 5% for pipelines dominated by scientific processing, while hashing-dominated cases are reported separately;
- external users can interpret all four outcomes correctly and author a basic contract from documentation in a measured usability study;
- no unresolved critical security findings remain at version 1.0;
- the public pilot can be reproduced by an independent team from the bundle and documented external resources.

Thresholds such as 95% and 5% are initial engineering hypotheses, not guaranteed facts. They must be revised transparently if pilot evidence shows that they are unrealistic or scientifically

inappropriate.

15.20 Kill criteria

A rigorous project must define when not to continue. ORIGIN Gate should merge into an established project, narrow its scope, or stop if any of the following holds:

- user research finds no consequential release decision that existing tools fail to support;
- Workflow Run RO-Crate or another project can incorporate the contract profile with less fragmentation;
- domain experts cannot define validators that are both useful and defensible;
- users frequently obtain false confidence or misunderstand `unknown`;
- the independent verifier exposes irreducible specification ambiguity;
- the maintenance burden of format adapters exceeds the value of the core;
- no external co-design partner is obtained by November 15, 2026;
- no independent pilot is completed by March 31, 2027;
- empirical results do not improve fault detection or diagnosis over simpler baselines;
- security or privacy requirements make portable evidence unacceptable for the intended application.

Stopping or contributing upstream under these conditions would be scientific success: it would prevent another redundant standard and produce useful evidence about what the community actually needs.

15.21 Final strategic conclusion

The path to an outstanding library is radical restraint. Build one release gate, one evidence-contract schema, one FITS-focused profile, one independent verifier, one adversarial corpus, and one external pilot. Reuse existing packaging, provenance, environment, and signature standards. Make `unknown` a first-class outcome. Prove only what the software can prove. Measure everything else.

If successful, ORIGIN Gate would not be famous because it stores more metadata. It would be valuable because it makes an important scientific release decision explicit, fail-safe, portable, and independently challengeable.

16 Second Red-Team Review: Could ORIGIN Gate Be Worthless?

The final redesign is more coherent than the original proxy, but coherence does not establish usefulness, originality, or adoption. This section evaluates the strongest case for abandoning the project. It focuses on the redesigned release gate rather than repeating the already established defects of the proxy code.

16.1 Brutally concise assessment

The problem is important, but the proposed solution may be unnecessary. Scientific data validation, workflow provenance, research-object packaging, dependency capture, signed attestations, archive schemas, CI release gates, and domain validators all exist. ORIGIN Gate currently contributes a proposed orchestration layer, a four-label outcome convention, and a packaging profile. None is yet shown to be scientifically novel, operationally superior, or requested by a real user.

There is no implementation of ORIGIN Gate. The only supplied code implements the discarded transparent-proxy idea and cannot serve as evidence that the new design works. The current manuscript is therefore a detailed design study and self-critique, not a software result, scientific result, or validated agency need.

16.2 The strongest redundancy argument

The proposed product sits at the intersection of several mature categories:

Need	Existing approach	Potential duplication by ORIGIN Gate
Data-quality contracts	Great Expectations checkpoints, Pandera schemas, TensorFlow Data Validation	Contract declaration, validation reports, and CI blocking [26, 27, 28]
Scientific file validation	FITSVERIFY, Astropy FITS verification, PDS4 Validate Tool	FITS/PDS schema, checksum, and structural checks [30, 31, 29]
Workflow provenance	AiiDA, noWorkflow, OpenLineage, W3C PROV	Inputs, outputs, runs, dependency graphs, and processing records
Research packaging	RO-Crate and Workflow Run RO-Crate	Portable bundle of data, metadata, workflow run, and provenance
Replay and environments	ReproZip, containers, lockfiles, Nix/Guix	Environment identity and reproduction references
Artifact attestations	in-toto, SLSA, DSSE, Sigstore	Signed statement connecting subjects, materials, process, and producer
ESA reproducible science	EarthCODE and Open Science Catalog	Publishing, validating, and sharing reproducible workflows and results [25]

If a team can obtain the desired behavior by combining an existing validator, a Workflow Run RO-Crate, and an in-toto attestation in a short CI script, a separate library and contract standard may create more fragmentation than value. The project must show that the integration itself solves a recurring failure that these tools cannot address cleanly.

16.3 The four outcomes are useful but probably not novel

Pass, fail, unknown, and error are sensible operational states. They are also close to ordinary testing and CI concepts such as success, assertion failure, skipped or inconclusive checks, and infrastructure error. The terminology does not constitute a scientific contribution.

The four states may also be inadequate:

- **unknown** conflates missing evidence, epistemic uncertainty, unavailable external resources, unsupported formats, and deliberately excluded dependencies.
- **error** conflates malformed input, verifier defect, timeout, permission failure, resource exhaustion, and incompatible version.
- Real assurance systems may require *not applicable*, *waived*, *stale*, *conflicting*, *pending review*, and *unauthorized*.
- Adding those states makes composition, visualization, policy, and user understanding substantially harder.
- A strict gate that blocks on every unknown may be bypassed or disabled by users under deadline pressure.
- A permissive gate that allows waivers may become ceremonial paperwork.

The proposed aggregate precedence also loses information. If one mandatory check fails and another verifier crashes, the formula returns **fail** and can hide the verifier error. A single aggregate label should never replace the complete vector of atomic outcomes. Different organizations may reasonably choose different precedence and waiver policies, undermining the aspiration to one universal gate semantics.

16.4 The central theorem risks being tautological

The proposed soundness statement is approximately: if the gate says pass, every mandatory check recorded in the contract passed. This is a valuable implementation invariant, but it follows almost directly from the definition of the gate. It says nothing about whether:

- the contract contains the right checks;
- the observation boundary includes the real causes of the product;
- validators are scientifically adequate;
- measurements and uncertainties are correct;
- inputs are authentic rather than merely hash-consistent;
- the pipeline is unbiased, safe, or logically valid;
- a malicious producer truthfully declared dependencies;
- the output supports the scientific claim attached to it.

The theorem may therefore be judged as software bookkeeping rather than a new scientific result. A cryptographically protected execution of insufficient checks remains insufficient.

16.5 Contracts can formalize error instead of eliminating it

The authority of the gate comes from the contract, but the contract is authored by humans or generated by software. A contract may omit a known failure mode, use the wrong physical unit, accept an unjustified tolerance, compare against a flawed reference, ignore covariance, or declare a dependency advisory to avoid blocking. Once formalized and signed, such an error may acquire greater institutional credibility than an informal script.

This produces a dangerous asymmetry:

The system can verify with high confidence that an inadequate policy was followed, while users may mistakenly interpret that as evidence that the product is valid.

Contract review, ownership, approval, versioning, and retirement may be more important than the gate engine. Those are organizational governance problems that a small Python package cannot solve.

16.6 The observation boundary may be an honesty label, not evidence

Boundary-relative completeness is more defensible than absolute completeness, but the boundary can become a disclaimer attached by the producer. The system cannot generally prove that all relevant dependency classes have been declared. A manual action, hidden service, stale cache, database query, notebook state, compiler flag, or instrument-side transformation may be absent from both the boundary and graph.

The coverage matrix may therefore describe what the producer believes or chooses to report rather than what actually influenced the result. Without independent observation—for example sandboxing, system-call capture, network isolation, or a controlled workflow engine—the “no silent completeness” guarantee is weaker than it first appears. The system can report no known unknowns while remaining ignorant of undeclared dependency classes.

16.7 Hashing may add cost without adding useful trust

Many archives and formats already store checksums. FITS supports checksum keywords; Astropy can verify them [31]. PDS product tooling checks referenced files and checksums [29]. Rehashing every large product may duplicate work and I/O.

A digest establishes equality to the hashed bytes, not provenance, authenticity, quality, legality, availability, or physical truth. If the original producer is compromised, both malicious data and its digest can be published together. If the artifact disappears, the digest verifies nothing useful. If semantically irrelevant metadata changes, the byte digest invalidates even when science does not; if a semantic profile is used, the profile may omit meaningful differences.

For petabyte-scale or remote products, verification cost, egress cost, archive load, and latency can exceed the practical value of an additional digest. Institutions may reasonably trust authoritative archive checksums rather than recompute them through ORIGIN.

16.8 The statistical layer remains fragile

Covariance-aware Mahalanobis validation is more defensible than independent scalar uncertainties, but it introduces new assumptions:

- the covariance matrix may be estimated poorly, stale, singular, or ill-conditioned;
- inversion may be numerically unstable and require regularization;
- the distance has a useful probabilistic interpretation only under suitable distributional assumptions;
- high-dimensional products may require dimensionality reduction that hides local failures;
- shared systematic errors can make a candidate and reference agree for the wrong reason;
- the reference may not be ground truth;
- repeated release checks create multiple-testing and selection effects;
- a threshold can be tuned after seeing results;
- spatial, temporal, and spectral correlations may not be representable by a tractable dense covariance matrix;
- model discrepancy and measurement uncertainty are not interchangeable.

Domain experts will still need to design and validate every scientifically serious check. The reusable gate engine contributes little to that intellectually difficult work.

16.9 The FITS flagship may be particularly redundant

NASA HEASARC’s FITSVERIFY already rigorously checks conformance with the FITS standard [30]. Astropy provides FITS verification behavior and checksum support [31]. Consequently, “validate a FITS file” is not a differentiating application.

The project would need to validate mission-specific scientific semantics that FITSVERIFY intentionally does not cover: calibration lineage, uncertainty, instrument quality policy, coordinate consistency, physical residuals, and release approval. Those semantics vary by mission and instrument. A generic contract may be too shallow, while a useful contract may require so much mission-specific work that the reusable library becomes a minor component.

16.10 Python may be the wrong center of gravity

A standards-compatible bundle should be language-neutral. Centering the identity of the project on a Python library may reduce its perceived relevance to Java, C++, R, Julia, shell, workflow-language, database, and cloud-native pipelines. If the real contribution is a contract schema and verifier semantics, Python is merely one producer SDK.

Python also creates maintenance concerns:

- interpreter and dependency support windows are short relative to mission lifetimes;
- packaging, native wheels, and platform compatibility change continually;
- scientific environments can be difficult to reproduce across CPU and GPU systems;
- a Python verifier may have a larger dependency and parser attack surface than a small compiled verifier;
- agency pipelines may forbid installing an unaudited package from a public package index.

If the Python SDK is removed and the remaining specification is still the main value, describing the work primarily as a Python library is misleading.

16.11 The independent verifier may not be independent enough

Two implementations can share one flawed specification, one mistaken test corpus, one cryptographic assumption, and one conceptual error. Agreement between them is necessary for interoperability but not proof of correctness. If both are written by the same team, use generated code, or rely on the same JSON and cryptographic libraries, common-mode failures remain.

True independence requires separate implementers, review, threat models, and test selection. That is expensive and unlikely before external adoption. Until then, “independently verifiable” means technically separable, not independently validated.

16.12 Standards-first can become standards-overload

Combining W3C PROV, IVOA Provenance, RO-Crate, Workflow Run RO-Crate, in-toto, DSSE, SLSA, OCI identities, institutional archive identifiers, and custom contracts can create an intimidating stack. Each standard has its own versioning, vocabulary, optional fields, tooling, and governance.

Mappings are rarely lossless. The same concept may have different identity, cardinality, temporal, or trust semantics across standards. A bundle that is technically conformant to several standards may be difficult for any one ecosystem to consume. The project could spend most of its effort maintaining adapters while delivering little new scientific value.

16.13 The release gate can become a bureaucratic bottleneck

Strict assurance improves quality only if checks are relevant, fast, stable, and trusted. Otherwise:

- false failures delay urgent releases;
- unknown outcomes block work for missing metadata that cannot be recovered;
- unstable statistical checks create flaky pipelines;
- teams add waivers until the gate is meaningless;
- experts spend more time maintaining contracts than improving science;
- developers route around the gate to meet operational deadlines;
- the report becomes another compliance artifact reviewed superficially;
- a central policy owner becomes a single organizational bottleneck.

The library can therefore reduce productivity and still fail to prevent the most important errors.

16.14 Security and privacy can reverse the value proposition

Evidence bundles can expose internal file paths, user identities, software versions, infrastructure topology, model identifiers, calibration schedules, operational thresholds, or sensitive data relationships.

A graph can reveal more than any individual artifact.

Redaction weakens independent verification; commitments to hidden low-entropy values can permit guessing attacks; signing introduces key-management obligations; and replayable environments can preserve vulnerable dependencies. An agency may prefer internal logs and access-controlled configuration systems over a portable bundle specifically because the bundle is portable.

16.15 Economic reasons it may have negative value

The relevant objective is net expected value, not technical elegance. A simplified model is

$$\mathbb{E}[U] = p_d C_f R_d - (C_i + C_c + C_m + C_s + C_o),$$

where p_d is the probability of a preventable defect, C_f its expected cost, R_d the fraction of such risk removed by the gate, and the subtracted terms are integration, computation, maintenance, security, and organizational costs. The gate is economically worthless when the checks duplicate existing controls, defects are rare or low consequence, users bypass the process, or ongoing maintenance dominates avoided failures.

The project currently has no measurements for any of these variables. Claims of real-world value are therefore hypotheses.

16.16 Why NASA may consider it completely unnecessary

NASA planetary archives already use the PDS4 information model, schemas, Schematron, discipline dictionaries, checksums, and the canonical PDS4 Validate Tool [29]. HEASARC already operates FITS verification tooling [30]. NASA Earthdata and individual missions have their own metadata, quality, processing, and release infrastructure.

NASA may reasonably ask:

1. Which documented agency requirement is unmet?
2. Why should an established validator call ORIGIN, rather than ORIGIN simply call the established validator?
3. Who will maintain the schema and security fixes for twenty years?
4. What authoritative system consumes the output bundle?
5. What failure has this tool detected that existing mission controls missed?
6. Why should a mission accept an external policy engine into a qualified release process?
7. What funded organization provides support, training, configuration control, and incident response?

Without compelling answers, ORIGIN is an additional dependency around systems NASA already owns and trusts. A NASA-affiliated researcher using it on public data would not mean NASA needs, endorses, or has adopted it.

16.17 Why ESA may consider it completely unnecessary

ESA EarthCODE already presents an environment for developing and publishing reusable code and workflows, validating outputs, and sharing reproducible results across platforms [25]. Mission consortia and contractors also operate under project-specific ECSS processes, configuration management, validation plans, and acceptance authority.

ESA may prefer to extend EarthCODE, Open Science Catalog metadata, an established workflow platform, or a project-specific validator. A generic external gate does not replace contractual requirements, independent verification and validation, nonconformance management, qualification evidence, or mission acceptance. Its four-state report has no authority unless an ESA project explicitly incorporates it into those processes.

The 2027 aspiration is especially weak. On August 19, 2026, fewer than seventeen months remain in 2027 and fewer than five months remain before it begins. A new design without implementation, external partner, security review, or pilot cannot credibly promise institutional use. Aggressive agency language may reduce trust by signaling unfamiliarity with adoption and assurance timelines.

16.18 Why the concept may not be a scientific contribution

The final concept can be decomposed into familiar ingredients:

- tests and contracts for data products;
- CI release gating;
- structured test results;
- workflow provenance;
- research-object packaging;
- content hashes;
- signed attestations;
- domain-specific statistical checks;
- explicit handling of incomplete evidence.

Integration can be excellent engineering without being new science. The four-state logic is elementary; the pass-soundness result is near-definitional; covariance-aware validation is established statistics; and the standards mappings are interoperability engineering. A paper needs a non-obvious result that changes what the community knows.

An empirical study could become scientific if it discovers, for example, that boundary-relative contracts reduce undetected scientifically meaningful defects by a substantial amount across independent pipelines without unacceptable false blocking. Until such data exist, the project is a proposal.

16.19 Why journals might reject the current manuscript immediately

In its present state, likely objections include:

- it contains no implementation or results for the final design;
- it is a long white paper rather than a conventional research article;

- it repeatedly redesigns and critiques itself, producing conceptual redundancy;
- the literature review is selective rather than systematic;
- several sources are standards pages, documentation, or preprints rather than a complete peer-reviewed foundation;
- research questions, hypotheses, sample sizes, and statistical analysis plans are not finalized;
- the proposed theorem may be considered trivial;
- the astrophysics example is hypothetical and contains no scientific result;
- there are no user interviews, external collaborators, benchmarks, or independent evaluators;
- the manuscript invokes NASA and ESA without a documented agency requirement or participation;
- broad claims span software engineering, astronomy, AI, security, reproducibility, and mission assurance without depth in each;
- proposed thresholds such as 95% detection and 5% overhead are arbitrary until justified and preregistered;
- no comparison shows superiority to a small CI script using existing tools;
- the absence of final code makes reproducibility impossible;
- prestigious general journals are unlikely to publish a software design without broad scientific impact or a major discovery.

The manuscript is currently valuable as an internal design and risk document. It should not yet be submitted as a research paper.

16.20 Why it might be completely worthless

ORIGIN Gate would be completely worthless, or worse than worthless, if most of the following are true:

- no real team has the proposed release problem;
- existing validators and workflow tools already solve it adequately;
- no archive, journal, workflow engine, or agency system consumes the bundle;
- contracts are copied mechanically and do not reflect scientific risk;
- users interpret `pass` as scientific truth;
- unknowns are waived routinely to keep pipelines moving;
- evidence capture misses undeclared dependencies;
- the tool detects only obvious schema errors caught elsewhere;
- hashing and packaging add delay and storage without preventing failures;
- the standards stack is too complex for users to understand;
- the independent verifier is not genuinely independent;
- mission-specific validators contain nearly all useful logic;
- maintenance ends after publication;
- portable evidence creates security or privacy risk;
- the project fragments rather than extends established standards;
- agency language is used for marketing without actual agency participation.

Under these conditions, the library would add ceremony, dependencies, and false confidence while consuming scarce research-software effort. The correct action would be to stop and contribute a small validator, schema, or test corpus to the existing system that users already employ.

16.21 Why it might nevertheless be unusually useful

The preceding critique does not prove that the project has no value. A narrow gap may remain: existing systems often validate data, record workflows, package research objects, or sign artifacts separately. A small layer that preserves the atomic evidence and produces a conservative release decision across these systems could be useful if it avoids becoming a new platform.

The differentiating value would be strongest when:

- an organization already has validators but lacks a portable, independently checkable policy result;
- missing evidence must block release rather than appear as a skipped test;
- scientific and supply-chain checks must be combined without losing their individual diagnostics;
- a calibration change requires cross-workflow impact analysis;
- a journal, archive, or collaboration agrees to consume the bundle;
- contracts and profiles are governed by real domain communities;
- the gate is substantially easier than each team building bespoke glue;
- empirical studies show fewer undetected consequential defects and faster diagnosis at acceptable cost.

The project may therefore be valuable as an extremely thin interoperability profile, conformance suite, and reference gate. Its chances decline as it adds platform features, custom vocabulary, or universal claims.

16.22 Current readiness score

The following scores describe the project on August 19, 2026, not its theoretical potential:

Dimension	Score / 5	Reason
Importance of the broad problem	5	Reproducibility, validation, provenance, and scientific-product integrity are consequential problems.
Clarity of final product thesis	3	The release-gate concept is coherent but still overlaps many established tools.
Demonstrated unmet user need	0	No interviews, requirements, or external pilot have been supplied.
Scientific novelty	1	Potential composition semantics exist, but no non-obvious result is established.
Final-design implementation	0	No ORIGIN Gate code exists.
Legacy proxy implementation	1	Basic demonstrations work, but provenance is unsound and the abstraction has been rejected.
Testability	4	A document-based gate and fixture corpus could be tested very thoroughly.
Security readiness	1	A threat model is proposed, but no implementation or audit exists.
Usability evidence	0	No user study or contract-authoring experiment exists.
NASA/ESA adoption readiness	0	No named partner, requirement, integration, or agency evaluation exists.
Journal submission readiness	0	There are no results for the final design.
Long-term sustainability	1	Governance ideas exist, but no organization, funding, or maintainer base exists.

These scores are intentionally severe. Improving them requires external evidence, not additional prose.

16.23 The one experiment that should decide whether to continue

Before implementing the full architecture, build the smallest possible gate around existing tools:

1. use FITSVERIFY or Astropy for structure and checksums;
2. use one mission-like scientific validator for units, calibration, and residuals;
3. package the workflow run using Workflow Run RO-Crate;
4. attach an in-toto-compatible attestation;
5. implement the four-outcome aggregation in fewer than one thousand lines;
6. compare it with an ordinary CI script on a blinded fault corpus;
7. give both approaches to independent astronomy users.

Continue only if the gate detects or explains consequential faults that the simpler approach misses, users understand its outcomes, integration remains small, and at least one external group wants to keep using it. If not, the experiment has answered the research question: a new library is unnecessary.

17 Final Recommendation: ORIGIN Phys

The preceding analysis leads to a stronger conclusion than another refinement of the provenance or release-gate idea. The project should change its primary concept. Provenance differencing with semantic comparators already exists [32]; scientific regression testing and metamorphic testing are established research areas [33, 34]; and recent work has moved toward domain-validity-gated metamorphic testing for scientific machine-learning systems [35]. Competing directly in those categories would require a substantial new algorithm and extensive differentiation.

The recommended library is instead:

ORIGIN Phys is a gradual physical-semantic type checker for scientific Python. Its objective is to make physically invalid programs detectable before execution.

It extends ordinary static types with physical meaning that is routinely lost in numbers and arrays: dimension, kind of quantity, unit, reference frame, epoch and time scale, tensor transformation law, and uncertainty or covariance structure. The name ORIGIN is retained because the checker tracks the semantic origin and valid transformation path of every typed physical value.

17.1 Why this problem is sharper

Unit libraries can reject addition of meters and seconds at runtime, but many severe errors are dimensionally valid. Torque and energy share the same SI dimensions; angular frequency and ordinary frequency differ by a factor of 2π ; position and displacement share units; geodetic and geocentric latitude are both angles; and two three-component arrays may refer to incompatible reference frames or epochs. Physical-type correctness research has already emphasized that unit compatibility alone cannot distinguish different kinds of quantity [36].

Space software adds further hazards:

- positions and velocities are meaningless without a frame and epoch;
- Earth-fixed and inertial frames cannot be mixed without a time-dependent transformation;
- UTC, TAI, TT, TDB, and mission elapsed time have different semantics;
- state covariance must transform with the appropriate Jacobian;
- attitude representations have ordering, handedness, normalization, and active/passive conventions;
- radiance, irradiance, flux, flux density, and counts can have compatible numerical shapes while representing different observables;
- AI models can receive arrays with the expected shape and dtype but the wrong units, frame, normalization, epoch, or physical interpretation.

NASA lessons explicitly emphasize persistent and accurate coordinate-frame records in mission development [39]. Static physical semantics cannot replace systems engineering, but it can prevent an important class of mistakes before a simulation, pipeline, or model is run.

17.2 What already exists and what must be new

Astropy, Pint, unyt, and similar libraries provide runtime physical quantities; Astropy provides sophisticated coordinate and time objects; `impunity` performs static unit analysis from annotations [37]; and recent work has integrated unit-aware arrays into high-performance AI-driven scientific computing [38]. Therefore, static units alone are not a sufficient contribution.

ORIGIN Phys must be distinguished by the combined type:

$$\tau = \text{mathsf{Phys}}\langle \mathbf{d}, \kappa, u, F, T, \rho, \mathcal{U} \rangle,$$

where

- $\mathbf{d} \in \mathbb{Q}^n$ is the physical-dimension vector;
- κ is a nominal kind of quantity, such as torque, energy, position, displacement, radiance, or angular frequency;
- u is a unit with scale and, where applicable, offset semantics;
- F is a coordinate, reference, or body frame;
- T records epoch and time-scale requirements;
- ρ describes representation and transformation behavior, including scalar, vector, covector, matrix, tensor variance, quaternion convention, or coordinate tuple;
- $\mathcal{U}$ describes uncertainty semantics, such as standard uncertainty, covariance, correlation, ensemble, or unknown uncertainty.

Any field may be the gradual unknown type $\star$. Unknown information permits incremental adoption but cannot silently satisfy an operation that explicitly requires known physical semantics.

17.3 Core typing judgments

The static checker should define a small formal calculus before supporting broad Python syntax. Illustrative judgments include:

Addition and subtraction. For $x : \text{Phys}\langle \mathbf{d}, \kappa, u_1, F, T, \rho, \mathcal{U}_1 \rangle$ and $y : \text{Phys}\langle \mathbf{d}, \kappa, u_2, F, T, \rho, \mathcal{U}_2 \rangle$,

$$x + y : \text{Phys}\langle \mathbf{d}, \kappa, u_c, F, T, \rho, \text{combine}_+(\mathcal{U}_1, \mathcal{U}_2) \rangle$$

is permitted only if u_1 and u_2 are convertibly coherent and the kinds, frames, epochs, and representations are compatible. Equal dimensions alone are insufficient.

Multiplication and division. Dimensions combine algebraically, but the result kind is obtained from a versioned kind registry or remains explicitly anonymous:

$$(\mathbf{d}_1, \kappa_1) \otimes (\mathbf{d}_2, \kappa_2) = (\mathbf{d}_1 + \mathbf{d}_2, \text{kindmul}(\kappa_1, \kappa_2)).$$

The checker must not invent a semantic kind from dimensional coincidence.

Frame transformation. A value in frame F_1 can become a value in frame F_2 only through a declared transformation morphism

$$\Phi_{F_1 \rightarrow F_2}(t, e) : \text{State}[F_1, t] \longrightarrow \text{State}[F_2, t],$$

where the required epoch t and external ephemeris or Earth-orientation evidence e are explicit. Renaming a frame annotation is never a valid transformation.

Covariance transformation. If $y = f(x)$ and J_f is the appropriate Jacobian, then a first-order covariance type follows

$$\Sigma_y = J_f \Sigma_x J_f^\top.$$

Dropping covariance, applying a coordinate transformation without transforming covariance, or attaching covariance expressed in a different frame produces a diagnostic unless explicitly acknowledged at an untyped boundary.

Time conversion. Conversion among time scales is modeled as an explicit operation with required context. Numeric addition cannot turn UTC into TDB, and a timestamp cannot be combined with a state vector whose epoch semantics are incompatible.

17.4 Soundness objective

For a formally specified subset of Python and NumPy expressions, the principal theorem should have the form:

If a closed expression is well typed under the ORIGIN Phys rules and evaluates without an external-library contract violation, then evaluation does not perform an undeclared dimension, quantity-kind, frame, epoch, representation, or uncertainty- structure mismatch covered by the type system.

This is stronger than “all configured checks ran.” It rules out a class of physically meaningless executions under stated assumptions. The proof should use a small core language with progress and preservation results and, if feasible, a mechanized model. Full Python remains gradual and conservative: unsupported dynamic behavior becomes unknown or requires a checked boundary contract.

17.5 Python representation

The public API should use standard annotations rather than runtime proxy objects:

```
1 from typing import Annotated
2 from numpy.typing import NDArray
3 from origin_phys import Phys, Frame, TimeScale, Covariance
4
5 PositionJ2000 = Annotated[
6     NDArray,
7     Phys(kind="position", unit="km", frame=Frame("J2000"),
8         epoch="t", representation="vector3"),
9 ]
10
11 VelocityJ2000 = Annotated[
12     NDArray,
13     Phys(kind="velocity", unit="km/s", frame=Frame("J2000"),
14         epoch="t", representation="vector3"),
15 ]
16
17 EpochTDB = Annotated[float, Phys(kind="time", unit="s",
18     time_scale=TimeScale("TDB"))]
19
20 def propagate(position: PositionJ2000,
21     velocity: VelocityJ2000,
22     epoch: EpochTDB) -> PositionJ2000:
23     ...
```

The checker runs through

origin-phys check src/

and integrates with editors, pre-commit, pytest, and CI. The annotated runtime value remains an ordinary NumPy array, so typed code pays no mandatory runtime wrapper or allocation cost. Optional boundary adapters inspect Astropy quantities, coordinates, times, FITS metadata, or SPICE-derived values when data enter typed code.

17.6 Diagnostics as a primary feature

An engineering-quality checker must explain physical errors in domain language:

```
EFRAE-004 at orbit.py:81
cannot add position[J2000, epoch=t0] to displacement[ITRF93, epoch=t1]

dimensions: compatible (length)
kinds:      compatible after explicit displacement conversion
frames:     incompatible (J2000 vs ITRF93)
epochs:     incompatible (t0 vs t1)
```

required:

1. propagate or transform both operands to a common epoch;
2. apply a declared J2000 -> ITRF93 transformation;
3. transform associated covariance with the same Jacobian.

The tool should never insert a scientifically consequential conversion silently. Suggested fixes may identify required operations but must require user confirmation.

17.7 Minimal architecture

```
origin_phys/  
|-- semantics.py      immutable physical-semantic type model  
|-- dimensions.py     exact rational dimension algebra  
|-- kinds.py          versioned nominal kind registry  
|-- frames.py         frame and transformation contracts  
|-- uncertainty.py    uncertainty and covariance rules  
|-- checker.py        expression and call checking  
|-- diagnostics.py    stable error codes and explanations  
|-- annotations.py    typing.Annotated public API  
|-- runtime.py        optional boundary validation  
|-- cli.py            check, explain, infer, benchmark  
'-- stubs/  
    |-- numpy/  
    |-- astropy/  
    '-- spiceypy/
```

Version 1 should exclude provenance storage, workflow execution, cryptographic attestations, automatic replay, web services, general AI governance, and arbitrary symbolic physics. The previous ORIGIN Gate can remain a later integration: its validators may consume ORIGIN Phys reports, but it is not the core product.

17.8 Space and astrophysics applications

Orbit and attitude software. Detect mixing of inertial and body-fixed coordinates, position and displacement, degrees and radians, epochs and time scales, active and passive rotations, quaternion orderings, and covariance frames.

Astrophysical catalogues and images. Check sky-coordinate frames, angular conventions, observation epochs, proper-motion epochs, wavelength and frequency representations, spectral and bolometric flux kinds, detector counts, calibrated rates, WCS axes, and uncertainty arrays.

Earth observation. Distinguish coordinate reference systems, altitude datums, radiance, irradiance, reflectance, brightness temperature, spatial resolution, acquisition time, and geolocation

covariance.

Scientific AI. Give models typed input and output signatures beyond shape and dtype. A model that expects normalized radiance in an Earth-fixed grid cannot silently receive raw counts in a different frame. Training and inference preprocessing can be checked against the same semantic interface.

Simulation and numerical physics. Detect illegal operations involving dimensional quantities, inconsistent state representations, missing nondimensionalization, covariance loss, and invalid coupling of fields defined on incompatible grids or frames.

17.9 Why this could be globally useful

The library addresses a recurring cross-domain failure mode without asking teams to replace workflow engines, archives, numerical libraries, or data formats. It can be adopted one function at a time, provides value in local development, produces ordinary CI diagnostics, and imposes no mandatory runtime object model. Its formal core is language-independent even though the first implementation targets Python.

The concept is also easy to explain:

Ordinary type checkers prevent adding a string to a number. ORIGIN Phys prevents adding a J2000 position at one epoch to an Earth-fixed displacement at another.

That message is more concrete than general claims about trust, evidence, or provenance.

17.10 Easy and rigorous testing

The test strategy should include:

1. **Typing golden files:** small programs with one expected diagnostic or one expected successful derivation.
2. **Algebra properties:** exact dimension algebra, unit-conversion composition, frame-path composition, and kind-registry consistency.
3. **Metamorphic checker tests:** renaming variables must not alter a judgment; coherent unit conversion must preserve it; changing only a frame must create the expected error.
4. **Mutation corpus:** automatically inject unit, kind, frame, epoch, time-scale, tensor, quaternion, and covariance faults into valid programs.
5. **Runtime differential tests:** compare static predictions with Astropy, Pint, SPICE, and NumPy boundary behavior.
6. **Parser and type-fuzzer tests:** generate Python ASTs and assert that the checker terminates without internal errors.
7. **Soundness-model tests:** exhaustively enumerate expressions in the small formal language and compare implementation judgments with the mechanized rules.

8. **Real-bug benchmark:** curate documented scientific-software bugs and realistic expert-written faults rather than relying only on synthetic mutants.
9. **Annotation-usability study:** measure time, errors, false positives, and perceived burden among scientists.
10. **Performance:** measure checker time and memory on realistic NumPy and Astropy codebases; runtime overhead should be zero when boundary checks are disabled.

Unlike full computational provenance, static judgments are deterministic and can be tested against finite typing rules. This makes the core unusually suitable for formal and property-based verification.

17.11 The research paper

The first paper should ask:

RQ1: Expressiveness.

What fraction of realistic space- and astrophysics-semantic faults can the combined type system express?

RQ2: Detection.

How many seeded and real faults are detected compared with mypy alone, runtime unit libraries, and static unit-only analysis?

RQ3: Precision.

What false-positive and unknown rates arise on valid scientific Python programs?

RQ4: Effort.

How much annotation and adaptation work is required?

RQ5: Performance.

What static-checking time and optional boundary-checking overhead are introduced?

RQ6: Human value.

Do diagnostics help scientists correct faults faster and more accurately than runtime failures or ordinary tests?

The research contributions should be:

1. a gradual physical-semantic type calculus combining quantity kind, unit, frame, epoch/time scale, representation, and uncertainty;
2. a soundness proof for a defined scientific-array core language;
3. a practical Python checker and typed library contracts;
4. SPACESEMBENCH, an open benchmark of physical-semantic software faults;
5. an empirical comparison and independent user study;
6. evidence about which physical semantics are worth encoding statically.

This is a genuine software-engineering or programming-languages contribution if the combined type system, proof, benchmark, and empirical results are new and strong. It becomes a scientific-computing contribution if it demonstrably prevents errors in real research code. It does not become a top-journal paper merely by invoking space missions.

17.12 Honest limitations

The final pivot does not eliminate every weakness:

- dynamic Python, reflection, monkey-patching, and native extensions limit static knowledge;
- NumPy shapes and dtypes are themselves incompletely expressible in today’s Python typing ecosystem;
- frames and time transformations depend on external ephemerides, leap-second data, and conventions that types can require but not validate completely;
- annotations can be wrong, incomplete, or intentionally bypassed;
- gradual unknowns trade guarantees for adoption;
- kind-of-quantity taxonomies can be disputed and domain-specific;
- affine and logarithmic units require special rules;
- nonlinear uncertainty propagation cannot be reduced generally to covariance propagation;
- type correctness does not establish algorithmic correctness, numerical stability, scientific validity, security, or mission safety;
- adoption depends on diagnostic quality and low annotation burden.

These limitations are narrower and easier to state than the completeness limits of universal provenance. The checker should fail conservatively and allow explicit, reviewable boundary casts rather than pretending complete knowledge.

17.13 NASA and ESA path for 2027

The credible objective is one non-flight pilot in a Python-based ground-science, simulation, or research pipeline. Potential integration partners include Astropy- based astronomy projects, SpiceyPy users, detector-simulation projects such as ESA’s Python-based Pyxel framework [40], university CubeSat teams, and mission- affiliated science groups.

No technical design can make NASA or ESA adoption “very, very likely.” By August 19, 2026, the project has no final implementation, benchmark, independent user, or agency partner. The fastest honest plan is:

Deadline	Required result
Sep. 15, 2026	Complete a systematic review of unit, physical-type, frame, and scientific-array typing; interview at least fifteen domain developers; freeze the minimal type tuple and ten priority fault classes.
Oct. 15, 2026	Release a checker prototype covering annotated scalar and NumPy arithmetic, units, quantity kinds, frames, and stable diagnostics.
Nov. 15, 2026	Publish the first SPACESEMBENCH corpus and demonstrate detection against mypy, Astropy/Pint runtime checks, and <code>impunity</code> . Obtain a named external co-design user.
Dec. 15, 2026	Release alpha software with editor and CI integration, property tests, mutation tests, documentation, and typed examples for Astropy or SpiceyPy.
Mar. 31, 2027	Complete an external pilot and publish false positives, annotation effort, missed faults, and checker performance.
Jun. 30, 2027	Complete the formal core, independent benchmark evaluation, and paper-ready study.
Dec. 31, 2027	Achieve a second external integration or narrow the project to the specific semantics users demonstrably need.

Use by a NASA- or ESA-affiliated research group in 2027 is possible only through early collaboration. Official operational adoption, flight qualification, or agency-wide standardization remains unrealistic on that timescale.

17.14 Global engineering strategy

To become durable rather than merely publishable:

- publish the type semantics independently of the Python implementation;
- use stable diagnostic codes and a machine-readable result format;
- make kind and frame registries extensible without allowing silent semantic redefinition;
- collaborate with, rather than fork, Astropy, Pint, NumPy typing, mypy, and SpiceyPy communities;
- maintain a strict compatibility and deprecation policy;
- provide a language-server experience with actionable explanations;
- establish governance involving programming-languages researchers and domain scientists;
- publish real bug cases, false positives, rejected rules, and unsoundness boundaries;
- keep the checker useful without a hosted service or proprietary registry;
- treat adoption into an established typing or scientific library as success, even if the independent ORIGIN brand eventually disappears.

17.15 Kill criteria for the final pivot

The project should stop or contribute upstream if:

- the literature review finds an existing system with the same combined semantics and practical integrations;

- domain developers do not report frame, epoch, kind, or covariance mistakes as consequential problems;
- annotations require more effort than users will accept;
- false positives or unknown results overwhelm actionable diagnostics;
- runtime libraries already detect nearly all benchmark faults at acceptable cost;
- the formal model cannot align with real NumPy and Astropy behavior without widespread unsound casts;
- no external co-design user is obtained by November 15, 2026;
- the benchmark contains only artificial faults and cannot attract independent contributions;
- the checker does not measurably improve fault detection or correction time.

17.16 Final verdict

Of the concepts considered in this paper, ORIGIN Phys has the best combination of narrow scope, immediate developer value, formal depth, testability, space-domain relevance, and realistic implementation size. It does not promise to prove science correct or capture every dependency. It targets a specific and damaging fact about scientific Python: arrays that look identical to the interpreter can mean radically different physical things.

The engineering masterpiece is therefore not a larger provenance platform. It is a small checker with a rigorous type theory, superb diagnostics, a difficult real-bug benchmark, near-zero adoption friction, and the intellectual honesty to reject programs it cannot understand.

18 Adversarial Review of ORIGIN Phys

The physical-semantic checker is the strongest concept considered in this paper, but it is still only a hypothesis. This section evaluates it as a skeptical programming-languages reviewer, scientific-software maintainer, mission engineer, Python typing expert, and potential institutional user would.

18.1 Summary judgment

ORIGIN Phys targets a genuine class of errors and is more focused, testable, and formally interesting than the previous concepts. Its best features are static feedback, no mandatory runtime wrapper, incremental annotations, domain-readable diagnostics, and the combined treatment of semantics that ordinary unit systems ignore.

Its principal weaknesses are equally serious. Units-of-measure typing is mature prior art; quantity-kind safety already exists in modern systems such as `mp-units` [44]; Astropy now offers unit-aware annotations [47]; `impunity` performs static unit checking in Python; and Python’s dynamic semantics make strong static guarantees difficult. The proposed combination of frames, epochs, representations, and covariance may be new, but it may also be an unmanageable product type whose annotations cost more than the errors it prevents.

There is still no ORIGIN Phys implementation. The uploaded code is the rejected provenance proxy.

Therefore, all claims about checker performance, usability, soundness, interoperability, and fault detection remain untested.

18.2 Complete inventory of strengths

Problem and product strengths

- The target problem is concrete: numbers and arrays discard physical meaning.
- The tagline is intelligible without explaining provenance theory.
- Detection before execution shortens feedback compared with runtime unit failures or late scientific review.
- The tool can be useful during local development without a database, network, workflow engine, archive, or hosted service.
- It does not require changing the runtime representation of every array.
- It can be introduced one function boundary at a time through gradual typing.
- Error messages can teach correct frame, time, covariance, and unit handling.
- Typed signatures double as precise, machine-checkable API documentation.
- The concept complements rather than replaces Astropy, Pint, NumPy, SPICE, and mission-specific runtime libraries.
- It has direct applications in orbit analysis, astronomy, Earth observation, detector simulation, numerical physics, and scientific AI.

Mathematical strengths

- Dimension vectors have a clean algebra over rational exponents.
- Nominal quantity kinds can distinguish physically different quantities with identical dimensions.
- Frames and time scales expose transformation preconditions that dimensions alone cannot express.
- Tensor representation and variance can prevent mathematically invalid coordinate operations.
- Covariance typing can connect a state representation to the uncertainty object that describes it.
- A small core language admits progress, preservation, and constraint-solving analysis.
- Gradual unknowns provide a principled boundary between proven and unproven semantics.
- Explicit conversions make assumptions and convention changes reviewable.

Engineering strengths

- `typing.Annotated` preserves ordinary runtime values and provides a standard place for tool-specific metadata.
- Static checks can impose zero runtime overhead when optional boundary checks are disabled.
- A CLI, editor protocol, and stable diagnostic codes fit existing development workflows.
- Golden typing judgments and mutation operators make regression testing straightforward.
- The core can be deterministic and offline.
- A standalone checker is not restricted to one host type checker's extension model.
- Typed stubs allow gradual integration with libraries that cannot be modified.
- The implementation can remain much smaller than a general provenance or workflow system.
- The specification can support later implementations in other languages.

Research strengths

- A combined physical-semantic calculus is a clearer research artifact than a collection of decorators.
- Soundness claims can be stated precisely for a bounded language.
- SPACESEMBENCH can become a reusable community resource even if the checker itself is superseded.
- Comparisons with unit-only systems yield falsifiable hypotheses.
- Annotation burden, false positives, and correction time can be measured.
- Real bug discovery can establish practical significance independently of formal novelty.
- The system creates a bridge between programming-languages research and scientific-software practice.

18.3 Prior art and similarity risk

The project enters a mature intellectual field. F# integrates units of measure into its type system and supports static checking and inference [41, 42]. Ada/GNAT provides compile-time dimensionality checking without runtime cost [43]. Modern C++ `mp-units` provides dimension, unit, and quantity-kind safety and is connected to ongoing C++ standardization work [44, 45]. Haskell and theorem-proving environments have long represented physical dimensions at the type level; formal ISQ/SI reasoning has also been mechanized in Isabelle/HOL [46].

Within Python and scientific computing:

- Astropy provides runtime quantities, physical types, coordinates, time scales, and unit-aware annotations [47].
- Pint and unyt provide established runtime unit systems.
- `impunity` performs definition-time static unit checking.
- the physical-type correctness proposal already argues that units and dimensions are insufficient to distinguish quantity kinds.
- SAIUnit integrates physical units with high-performance scientific and AI array computation.

The originality cannot therefore be “static physical units for Python,” “unit-aware arrays,” or “quantity-kind safety.” The only plausible novelty is the sound, practical combination of quantity kind, frame, epoch/time scale, representation, and uncertainty semantics in gradual Python code. A systematic review may find that even parts of this combination exist in dependently typed, robotics, geospatial, or aerospace systems. If so, the contribution narrows further to usability and empirical evidence in scientific Python.

18.4 Fundamental type-theoretic difficulties

The semantic product may be too large

The proposed type has at least seven axes. Compatibility is not a simple equality test:

- units can be exactly convertible, affine, logarithmic, contextual, or convention-dependent;
- quantity-kind taxonomies may form graphs rather than a clean nominal tree;
- frames may be parameterized by body, observer, origin, orientation, epoch, and reference data;

- time scales can require historical tables and relativistic transformations;
- representation compatibility depends on basis, ordering, handedness, and active/passive convention;
- uncertainty semantics can be scalar, covariance, ensemble, interval, distribution, or model-dependent.

The cross-product can produce types too verbose for humans and constraint solving too expensive for a responsive editor. Most values will have partial information, causing frequent gradual unknowns that weaken the guarantee.

Quantity kinds lack a universal algebra

Dimension multiplication is mechanical; semantic kind multiplication is not. Multiplying force by distance may represent work, energy transfer, or torque magnitude depending on geometry and intent. Dividing energy by time may be power, but not every dimensionally equivalent expression should acquire that nominal kind. A global registry risks encoding disputed ontology as compiler behavior.

If kind inference is conservative, many valid expressions require annotations or casts. If it is aggressive, the checker silently invents meaning. Domain-specific registries can conflict, and schema evolution can change whether previously valid code type-checks.

Frames are values as well as types

“J2000” is not always enough to identify a transformation. Real coordinate objects can depend on observer location, obstime, equinox, ephemeris, Earth-orientation parameters, body-fixed kernels, instrument alignment, and calibration versions. These are often runtime values, not compile-time constants.

A static type can prove that a transformation function was called with something typed as required context; it generally cannot prove that the ephemeris is correct, current, mutually compatible, or applicable to the observation. Parameterizing types by runtime epochs and frames approaches dependent typing that mainstream Python checkers do not support.

Time semantics are exceptionally difficult

UTC is discontinuous because of leap seconds; UT1 depends on Earth-rotation data; TDB and TT transformations involve conventions and approximations; mission elapsed time depends on a mission epoch and clock model. Static types can distinguish labels but not calculate or validate all transformations without runtime data.

If types include exact epochs, almost every timestamp has a distinct value-indexed type. If they include only symbolic epoch variables, aliasing and equality of those variables become a constraint problem. If epoch is omitted, many frame guarantees become too weak.

Uncertainty is not a single compositional type

The covariance rule $J\Sigma J^\top$ is a local linear approximation. Nonlinear, multimodal, bounded, heavy-tailed, correlated, and systematic uncertainties may require ensembles, unscented transforms, sampling, or domain models. A type can ensure that covariance is transformed, but it cannot generally ensure the transformation is an adequate description of uncertainty.

It is also unclear whether uncertainty belongs in the static type or in runtime data. Two values may have the same uncertainty representation but different matrix dimensions, conditioning, rank, or correlation structure. Encoding all of this statically could make the checker unusable.

Gradual typing weakens strong slogans

The slogan “make physically invalid programs detectable before execution” is more honest than “make them impossible,” but still risks overstatement. Any use of **Any**, unknown metadata, unsupported function, dynamic import, unchecked cast, external native call, or untyped boundary can allow invalid semantics to pass. Python’s typing specification permits tools that do not understand **Annotated** metadata to ignore it and treat only the base type [48].

The guarantee is therefore tool-local: code checked by ORIGIN Phys under a specific configuration satisfies the supported rules, assuming its stubs and annotations are correct. Ordinary mypy or Pyright users may receive no physical- semantic protection at all.

18.5 Python and NumPy implementation problems

- Python permits rebinding, reflection, monkey-patching, dynamic attribute creation, metaclasses, runtime code generation, and conditional imports.
- **Any** and casts provide easy escape routes.
- Decorators can replace a function with behavior unrelated to its annotated signature.
- Overloads and protocols may lose or widen physical metadata.
- Existing type checkers are not required to preserve arbitrary **Annotated** metadata through type operations.
- A standalone checker must duplicate Python name resolution, generics, overload selection, type narrowing, and incremental analysis, or rely on another checker’s internal representation.
- A host-checker plugin inherits that checker’s release cadence, plugin API, inference behavior, and limitations.
- Supporting both mypy and Pyright may require two substantially different integrations.

NumPy compounds the difficulty:

- ufuncs have large, overloaded, extensible dispatch surfaces;
- broadcasting changes shapes and semantic axes;
- reductions remove axes whose physical meaning may matter;
- **einsum**, tensor contraction, stacking, indexing, views, structured arrays, masked arrays, and record arrays require distinct rules;
- **where**, **out**, in-place operations, and dtype coercion alter result semantics;
- arrays frequently encode multiple physical fields or axes in one object;

- shape typing itself remains incomplete because of Python type-system and checker limitations [49];
- third-party array libraries implement NumPy-like operations with different semantics.

A scientifically meaningful checker may require hundreds or thousands of carefully maintained function contracts. Incorrect stubs create either false confidence or false alarms.

18.6 Runtime boundary and data-ingestion problems

Many physical errors originate in data rather than source annotations. A FITS header may contain the wrong unit, an ephemeris file may be stale, columns may be swapped, a model checkpoint may expect different normalization, or an instrument pipeline may label a frame incorrectly. Static typing sees the declared boundary type, not whether runtime bytes satisfy it.

Optional boundary validation is therefore necessary, but it reintroduces runtime cost, format adapters, error handling, external context, and duplicated metadata. Annotations may drift from FITS, database, schema, or model metadata. The checker must choose an authority or report conflict; neither choice is purely static.

18.7 Usability and adoption weaknesses

- The annotations shown in the proposal are long and visually dominant.
- Scientists may prefer descriptive variable names and tests to seven-axis types.
- Legacy code often uses raw arrays pervasively, requiring extensive boundary annotation.
- False positives in mathematically valid expert code can cause users to add broad casts or disable the checker.
- Frequent unknown results may make the tool feel weaker than its marketing.
- Domain ontologies and frame registries require governance that most users do not want to understand.
- Error messages can recommend a syntactically valid conversion that is scientifically inappropriate.
- Editor latency can destroy perceived value even when batch checking works.
- Teams may resist introducing a specialized checker that new staff must learn and CI must maintain.
- Library authors must publish accurate semantic stubs before downstream code receives much benefit.

The annotation burden must be measured against actual prevented faults, not judged from attractive examples.

18.8 Security and assurance limits

Static physical typing is not a security mechanism. It does not prevent malicious data, code execution, dependency compromise, numerical instability, race conditions, memory errors in native extensions, denial of service, privacy leakage, or unauthorized operations. It does not provide requirements traceability, configuration control, independent verification, or a safety case.

Incorrect stubs or a compromised checker can suppress diagnostics. A signed release of the checker does not validate the scientific ontology. Mission assurance would need the checker version,

configuration, registries, stubs, suppressions, and output placed under configuration control and independently qualified for its intended role.

18.9 Why it might not be useful

ORIGIN Phys may be unnecessary when teams already represent values with Astropy `Quantity`, `SkyCoord`, `Time`, SPICE objects, domain classes, or strongly typed C++/Ada interfaces. Runtime classes can carry the actual epoch, frame, unit, and metadata rather than approximating them statically. Thorough tests, interface-control documents, peer review, and existing mission validators may already control the relevant risks.

The checker is also less useful for exploratory notebooks, one-off analyses, I/O- dominated scripts, or code where physical semantics are determined only at runtime. If the targeted mistakes are rare, caught quickly by existing libraries, or low consequence, annotation and maintenance cost may exceed avoided failures.

It may become a research prototype used only in its own benchmark: impressive type rules, few real annotations, obsolete stubs, and no maintained editor integration. In that case the library has little practical value even if the paper is accepted.

18.10 Why it might be very useful

The checker is most valuable for long-lived numerical codebases with many raw arrays, multiple coordinate systems, evolving calibration, large teams, AI model interfaces, and expensive or delayed failures. It could:

- detect incompatible frames before a costly simulation or processing run;
- prevent unit- and kind-correctness mistakes that runtime dimension checks miss;
- document API assumptions precisely for collaborating teams;
- enforce consistent AI preprocessing and output semantics;
- ensure covariance follows state transformations;
- reveal ambiguous code during review even when it cannot prove correctness;
- provide stable CI checks with no production runtime dependency;
- create a reusable benchmark of scientific semantic faults;
- encourage scientific libraries to expose more precise semantic interfaces.

Its value may come less from rejecting many bugs and more from forcing teams to make critical conventions explicit.

18.11 Why it might not be a genuine scientific contribution

A skeptical reviewer can decompose the work into old ingredients: units-of-measure types, nominal quantity types, phantom or refinement types, frame-tagged vectors, time types, tensor types, covariance propagation, Python annotations, and a linter. Combining known type parameters in one tuple is not automatically novel.

The formal soundness theorem may concern an artificial language far simpler than NumPy and Astropy. If the implementation handles real code through unsound stubs, casts, and ad hoc rules, the theorem may have little practical significance. Conversely, if the formal rules dominate the work but adoption is poor, the paper may lack scientific-software impact.

SPACESEMBENCH can also be criticized if faults are synthetic, selected by the authors, easy for the checker by construction, or not representative of real mission defects. A benchmark is a contribution only when independently curated, versioned, diverse, and adopted beyond the originating project.

18.12 Why journals might reject it

Likely reasons include:

- prior art already covers unit inference and quantity-kind safety;
- the novelty of adding frames, epochs, and covariance is not demonstrated by a systematic review;
- the formalism is a straightforward product of existing type indices;
- the soundness theorem applies only to a toy language;
- the Python implementation is incomplete or relies heavily on casts;
- no realistic library stubs or integrations are available;
- benchmark mutations mirror the checker’s rules and inflate detection;
- no real historical bugs are detected;
- annotation effort and false positives are unacceptable;
- the user study is small, unblinded, or performed only with students;
- comparisons omit F#, C++ `mp-units`, Ada, Astropy annotations, `impunity`, or other relevant systems;
- agency relevance is speculative and unsupported by collaborators;
- the manuscript is currently a design history rather than a focused research article;
- broad-journal significance is not established by a Python checker alone.

The current long-form design manuscript should be split before submission. The paper should contain the final type system, related work, implementation, evaluation, and limitations. The discarded provenance and gate designs belong in a technical report, thesis chapter, or project history.

18.13 Why NASA and ESA might not need it

NASA and ESA do not lack awareness of units, coordinate frames, time systems, or configuration control. Flight and operational software frequently uses languages, processes, interface-control documents, data standards, reviews, tests, and mission-specific frameworks outside ordinary Python typing. Existing coordinate and time libraries already centralize transformations, while domain classes can make misuse difficult at runtime.

Agency programs may reject ORIGIN Phys because:

- critical code is not Python or is generated from models;
- Python is confined to analysis where failures are already caught by experts;
- Astropy, SPICE, mission frameworks, C++ `units`, or Ada dimensional checking already provide sufficient protection;

- annotations duplicate authoritative interface-control and data-schema metadata;
- the checker and registries create another qualification and maintenance burden;
- static approximations cannot validate runtime ephemerides, clock data, calibration, or observation metadata;
- false positives could disrupt established pipelines;
- the external project lacks long-term funding, support, cybersecurity review, and configuration governance;
- one semantic checker addresses only a small fraction of mission-software risk;
- adoption by a research group would not justify integration into operational agency processes.

An agency may prefer a few additional checks in Astropy, SPICE wrappers, Pyxel, or an existing static-analysis system rather than a separate checker.

18.14 Why NASA and ESA might need it

The word “need” is too strong without requirements evidence, but a useful gap can exist in Python-based ground science, instrument simulation, rapid mission analysis, AI prototypes, and research pipelines. These environments often combine raw NumPy arrays with domain libraries, allowing semantic information to disappear between API boundaries.

An agency-affiliated team might value ORIGIN Phys if it:

- finds real frame, epoch, unit-kind, or covariance defects missed by current tests;
- integrates with existing Astropy/SPICE types rather than replacing them;
- runs offline in CI with negligible production impact;
- uses project-controlled registries and stable diagnostics;
- helps review AI model input and output contracts;
- reduces onboarding and interface misunderstandings across institutions;
- supplies evidence that its false-positive and annotation costs are low;
- is maintained by a credible consortium with agency-affiliated contributors.

The plausible first use is advisory static analysis for non-flight code. If it consistently catches consequential defects, a project could later make selected diagnostics mandatory.

18.15 Why it might be completely worthless

ORIGIN Phys would be worthless or harmful if:

- its combined semantics already exist in another practical system;
- it detects only dimensional unit errors that runtime libraries already catch;
- most real values reach the checker through `Any` or untyped arrays;
- annotations are longer and harder to maintain than the scientific code;
- frame and epoch semantics remain too runtime-dependent for useful static judgments;
- the kind registry encodes arbitrary or disputed classifications;
- false positives encourage blanket ignores and casts;
- stubs drift from library behavior and create false confidence;
- no real historical bug is found;

- SPACESEMBENCH consists only of checker-friendly mutants;
- editor integrations become unmaintained;
- the formal proof covers almost none of the practical implementation;
- users prefer contributions directly to Astropy, Pint, mypy, or NumPy typing;
- NASA/ESA language is used without participation or operational relevance;
- maintenance stops after paper publication.

Under those conditions, the project adds annotation ceremony and a false aura of physical correctness while consuming effort better spent strengthening existing libraries.

18.16 Updated readiness score

Dimension	Score / 5	Assessment on August 19, 2026
Importance of target errors	4	Unit, frame, time, representation, and covariance mistakes can be consequential.
Clarity of product thesis	4	The static physical-semantic checker is focused and memorable.
Demonstrated novelty	1	The combined semantics may be novel, but prior-art review and implementation are incomplete.
Formal feasibility	3	A small calculus is feasible; faithful Python/NumPy coverage is much harder.
Implementation readiness	0	No ORIGIN Phys code exists.
Testability	4	Typing rules and mutation operators are highly testable.
Real-bug evidence	0	No historical or independent bugs have been demonstrated.
Usability evidence	0	Annotation burden and false positives are unknown.
NASA/ESA adoption readiness	0	There is no partner, requirement, pilot, or agency evaluation.
Publication readiness	0	There are no implementation results or final proof.
Long-term sustainability	1	A governance plan exists only as prose.

The concept has improved, but readiness has not. More design text cannot raise the scores that require code, users, defects, experiments, and independent review.

18.17 The decisive feasibility experiment

Before building the full type system:

1. select one real 2,000–10,000 line Astropy or SpiceyPy codebase;
2. annotate only positions, velocities, times, frames, and covariance;
3. implement checks for arithmetic, calls, one explicit frame transform, and covariance transformation;

4. seed a blinded corpus of at least thirty faults, with one third written by an independent domain expert;
5. compare mypy, Astropy runtime behavior, `impunity`, and ORIGIN Phys;
6. measure additional detections, missed faults, false positives, annotations, checker time, and correction time;
7. ask the external maintainer whether the annotations should remain in the codebase after the experiment.

Continue only if ORIGIN Phys detects consequential non-unit semantic faults that the baselines miss, annotation effort is acceptable, diagnostics lead to correct fixes, and the maintainer voluntarily retains the tool. Otherwise contribute the useful rules or stubs to an established project and stop creating a separate library.

19 Ultimate Narrowing: ORIGIN Frame

The direct-prior-art review changes the recommendation again, and this correction is scientifically necessary. PHYSFRAME already developed a frame-semantics type checker for C/C++ ROS software, evaluated 180 projects, and reported 154 true positives [50]. The `wave_geometry` C++ library also provides compile-time coordinate-frame semantics [51]. Frame typing itself is therefore not a new idea.

The only defensible independent project is a narrower extension into an inadequately served environment:

ORIGIN Frame is a gradual static analyzer for frame, epoch/time-scale, representation, and covariance consistency across NumPy, Astropy, and SpiceyPy boundaries.

It delegates dimensions and units to established libraries. It does not claim to invent frame safety. Its research question is whether the dynamic, array-oriented, runtime-context-heavy semantics of space-science Python require and benefit from a new analysis beyond PHYSFRAME, runtime domain objects, and existing unit systems.

19.1 Precisely bounded novelty claim

The project may claim novelty only if it demonstrates all of the following:

1. a Python/NumPy analysis that preserves frame meaning through array operations and cross-library APIs;
2. symbolic epoch and time-scale obligations that prevent invalid static combinations without pretending runtime tables are known;
3. a type relationship binding covariance to the exact state representation, frame, epoch, ordering, and transformation Jacobian it describes;
4. useful diagnostics on real Astropy- or SpiceyPy-oriented code with tolerable annotation burden;
5. detection of consequential faults not caught by runtime domain classes, tests, ordinary typing, unit-only checking, or straightforward adaptation of PHYSFRAME.

If one of these conditions fails, the appropriate contribution is an extension to an existing project, not a new general library or a broad novelty claim.

19.2 Minimal semantic model

Version 1 should use a much smaller type than ORIGIN Phys:

$$\tau = \text{State} \langle F, E, S, R, U \rangle,$$

where F is the reference frame, E the epoch variable, S the time scale, R the state representation and ordering, and U the uncertainty relationship. Units remain part of an Astropy, Pint, or other established quantity type rather than an ORIGIN registry.

Examples of R include Cartesian position, Cartesian position–velocity state, spherical coordinates, active quaternion with scalar-first ordering, passive rotation matrix, and astrometric parameters. Examples of U include no uncertainty, standard deviations aligned with state components, covariance for the complete state ordering, and ensemble uncertainty.

The checker should enforce only high-value rules:

- state arithmetic requires compatible frames and epochs;
- a transformation changes the frame only through a declared transform contract;
- time-dependent transforms require an epoch of the required time scale;
- propagation changes epoch through a declared dynamical propagator;
- representation conversion is explicit and cannot be confused with frame transformation;
- covariance cannot be combined with a differently ordered or framed state;
- a nonlinear transform must declare how uncertainty is propagated or return an explicitly unknown uncertainty relationship;
- untyped or runtime-dependent boundaries remain visible in diagnostics.

19.3 Illustrative Python interface

```

1 from typing import Annotated, TypeVar
2 from numpy.typing import NDArray
3 from origin_frame import State, Covariance, frame_transform
4
5 T = TypeVar("T") # symbolic epoch identity
6
7 J2000State = Annotated[
8     NDArray,
9     State(frame="J2000", epoch=T, time_scale="TDB",
10         representation="cartesian_pv"),
11 ]
12
13 J2000Cov = Annotated[
14     NDArray,
15     Covariance(of=J2000State, ordering="x,y,z,vx,vy,vz"),
16 ]

```

```

17 @frame_transform(source="J2000", target="ITRF93",
18                 epoch_scale="TDB", jacobian=True)
19
20 def to_earth_fixed(state: J2000State, covariance: J2000Cov,
21                   epoch: T):
22     ...

```

The design must avoid stringly typed production APIs where possible. The example uses strings for readability; the implementation should intern validated frame, time-scale, and representation declarations and reject ambiguous aliases.

19.4 Critical implementation decision

Do not build a full replacement for mypy or Pyright. The first prototype should use one host checker’s typed abstract syntax tree, supplemented by a small dataflow analysis for semantic metadata. A standalone checker should be considered only if the host extension model makes the essential analysis impossible.

The codebase should remain approximately:

```

origin_frame/
|-- semantics.py      frame, epoch, representation, covariance types
|-- contracts.py     Astropy and SpiceyPy API contracts
|-- analysis.py      semantic dataflow and constraint solving
|-- checker_plugin.py host-checker integration
|-- diagnostics.py   stable codes and domain explanations
|-- runtime.py       optional boundary introspection
|-- cli.py           check and benchmark commands
'-- benchmark/       mutation operators and corpus harness

```

The checker should initially support fewer than twenty operations and support them correctly. Unsupported NumPy behavior must produce a visible loss-of-semantics diagnostic rather than silently degrading to an apparently safe array.

19.5 The genuine research program

The strongest paper is an empirical and formal extension of prior work, not a vision paper. It should contain:

1. a systematic review of frame-safety and physical-semantics analyses;
2. a formal calculus for symbolic epochs, representations, and covariance- coupled frame transformations;
3. a prototype integrated with one mainstream Python checker;
4. SPACEFRAMEBENCH, including independently sourced real defects and expert-authored mutants;
5. a comparison with PHYSFRAME concepts, runtime Astropy/SPICE objects, ordinary typing, and unit-only analysis;

6. evaluation on at least three independently maintained Python codebases;
7. a blinded user study measuring correction time and incorrect suggested fixes;
8. an external replication package and negative results.

Potential technical contributions include a gradual treatment of runtime epoch parameters, preservation of semantics through scientific-array operations, and static coupling of covariance to state representation. These contributions must be shown to be nontrivial relative to PHYSFRAME and compile-time geometry libraries.

19.6 The 10/10 engineering rule

A 10/10 score cannot be declared in advance. It must be earned across independent dimensions:

Dimension	Evidence required before a strong claim
Correctness	Formal core, golden judgments, property tests, mutation tests, and no known unsound silent downgrade in supported operations
Novelty	Systematic review and explicit comparison with PHYSFRAME, <code>wave_geometry</code> , Astropy, unit systems, and relevant static analyses
Usefulness	Retention by external maintainers after a real pilot
Usability	Measured annotation effort, diagnostic comprehension, and correction time with domain professionals
Performance	Editor-responsive incremental checking and measured CI cost
Scientific relevance	Real or independently written space-science defects that baselines miss
Security	Offline operation, dependency review, signed releases, and no execution of analyzed code by default
Sustainability	Multiple maintainers, stable governance, funding plan, and upstream collaboration
Agency relevance	Named partner, documented requirement, pilot report, and integration owner
Publication strength	New result, complete baselines, external replication, and conclusions proportional to evidence

Failure in one dimension cannot be hidden by excellence in another. A beautiful formalism with no users is not a 10/10 engineering product; a useful linter with no new knowledge is not a 10/10 research paper.

19.7 Agency adoption strategy

The only credible way to make 2027 NASA/ESA-affiliated use reasonably likely is to obtain the user before building the generalized tool. By September 2026, the project should identify one Python ground-software team with an actual frame/epoch defect history and co-design the first rules around

its code. The partner must choose the benchmark cases, acceptable annotation burden, deployment environment, and success criteria.

The tool should be offered first as advisory CI analysis. It must run offline, require no source upload, generate machine-readable diagnostics, and integrate with the team’s existing Astropy/SPICE conventions. Agency-wide branding, flight-use claims, and generic safety claims should be prohibited without formal authorization and qualification evidence.

Even under ideal execution, use by one NASA- or ESA-affiliated research group in 2027 is a stretch objective rather than a high-probability consequence of technical design. Institutional adoption is controlled by relationships, requirements, funding, assurance, and maintenance commitments that the library cannot determine.

19.8 Go/no-go experiment

The entire independent project should be decided by a twelve-week experiment:

1. reproduce PHYSFRAME’s relevant analysis concepts and document what changes for scientific Python;
2. obtain one external Astropy/SpicePy codebase and domain maintainer;
3. implement frame, symbolic epoch, representation, and covariance checks for its five most important operations;
4. create thirty blinded faults, at least fifteen chosen by the external expert;
5. compare ordinary tests, runtime libraries, ordinary typing, a manually adapted PHYSFRAME-style baseline, and ORIGIN Frame;
6. measure unique detections, false positives, annotation time, checker time, correction accuracy, and whether annotations are retained;
7. publish the complete result even if the tool provides no advantage.

Proceed only if the tool uniquely detects consequential defects, the maintainer wants it retained, and the epoch/covariance analysis is demonstrably more than a straightforward port of prior work. Otherwise upstream the benchmark and useful contracts, then end the independent library.

19.9 Ultimate conclusion

There is no intellectually honest path to guaranteed perfection, guaranteed prestigious publication, or highly probable agency adoption from concept design alone. The best available path is a narrowly scoped, prior-art-aware experiment with an external user and a willingness to terminate the project.

If that experiment succeeds, ORIGIN Frame could become an excellent Python static-analysis tool and a strong research contribution. If it fails, discovering that existing tools are sufficient is the correct scientific result.

20 Publication and Adoption Strategy

A library announcement is insufficient for a leading venue. The first defensible paper is a programming-languages or software-engineering paper combining:

1. the gradual frame/epoch/representation/covariance calculus;
2. a soundness result for the supported core language;
3. the practical Python checker and diagnostics;
4. SPACEFRAMEBENCH with expert-authored and real faults;
5. comparison with PHYSFRAME concepts, runtime Astropy/SPICE systems, ordinary typing, and unit-only checking;
6. an independent user study and at least one real scientific-code case study.

Plausible first venues include PLDI, OOPSLA, ICSE, or a leading software-engineering journal if the formalism and empirical evidence meet their standards. A later computational-science or astronomy paper should report consequential faults found in real research software or a substantive analysis made safer by the checker. A prestigious broad scientific journal becomes plausible only after independent, cross-domain impact; it should not be the initial publication target.

21 Roadmap to a 2027 Pilot

The controlling roadmap is the go/no-go ORIGIN Frame experiment above. Because August 19, 2026 is the present date, a credible 2027 pilot requires a named external co-design user by November 15, 2026 and an alpha release by December 15, 2026. Experimental use by a NASA- or ESA-associated research group during 2027 is possible but cannot be promised. Flight-critical deployment, agency-wide standardization, and qualification remain outside the credible 2027 scope.

22 Success Criteria

The project succeeds when independent scientists can annotate state-vector and coordinate interfaces incrementally, receive precise diagnostics for frame, epoch, representation, and covariance faults before execution, and correct those faults with substantially less effort than with ordinary typing and runtime checks alone. The formal rules, implementation, benchmark, and empirical results must agree; unsupported behavior must remain explicit rather than being treated as safe.

The final conceptual change is:

Not a Python object that remembers every operation, not a new provenance platform, and not another unit library, but a narrowly evaluated static analysis that prevents frame-, epoch-, representation-, and covariance-incompatible states from being combined silently.

References

- [1] World Wide Web Consortium, *PROV Overview: An Overview of the PROV Family of Documents*, W3C Working Group Note, 2013. <https://www.w3.org/TR/prov-overview/>
- [2] International Virtual Observatory Alliance, *IVOA Provenance Data Model*, Version 1.0, 2020. <https://www.ivoa.net/documents/ProvenanceDM/>
- [3] G. Pizzi, A. Cepellotti, R. Sabatini, N. Marzari, and B. Kozinsky, “AiiDA: automated interactive infrastructure and database for computational science,” *Computational Materials Science*, vol. 111, pp. 218–230, 2016.
- [4] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “noWorkflow: A tool for collecting, analyzing, and managing provenance from Python scripts,” *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1841–1844, 2017. <https://github.com/gems-uff/noworkflow>
- [5] OpenLineage Project, *OpenLineage Specification*. <https://openlineage.io/docs/spec/>
- [6] NASA Science Mission Directorate, *Data and Computing Strategy 2025–2030*, 2025. <https://science.nasa.gov/open-science/data-and-computing-architecture-study/>
- [7] NASA Science Mission Directorate, *Scientific Information Policy for the Science Mission Directorate, SPD-41a*, 2022. NASA SPD-41a policy page
- [8] European Cooperation for Space Standardization, *ECSS-Q-ST-80C Rev.2: Space Product Assurance—Software Product Assurance*, 2024. ECSS-Q-ST-80C Rev.2 landing page
- [9] S. Soiland-Reyes et al., “Packaging research artefacts with RO-Crate,” *Data Science*, vol. 5, no. 2, pp. 97–138, 2022. <https://www.researchobject.org/ro-crate/>
- [10] S. Leo et al., “Recording provenance of workflow runs with RO-Crate,” 2024. Workflow Run RO-Crate specification and resources
- [11] F. Chirigati, D. Shasha, and J. Freire, “ReproZip: Using provenance to support computational reproducibility,” in *Proceedings of the 5th USENIX Workshop on the Theory and Practice of Provenance*, 2013. <https://www.usenix.org/conference/tapp13/technical-sessions/presentation/chirigati>
- [12] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *28th USENIX Security Symposium*, 2019. <https://in-toto.io/>
- [13] Open Source Security Foundation, *Supply-chain Levels for Software Artifacts: Provenance*. <https://slsa.dev/provenance>
- [14] H. Suetake et al., “A workflow reproducibility scale for automatic validation of biological interpretation results,” 2023. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10164546/>
- [15] F. Schintke et al., “Validity constraints for data analysis workflows,” 2024. <https://arxiv.org/abs/2305.08409>
- [16] L. Pouchard, K. G. Reyes, F. J. Alexander, and B.-J. Yoon, “A rigorous uncertainty-aware quantification framework is essential for reproducible and replicable machine learning workflows,” 2023. <https://arxiv.org/abs/2301.05763>

- [17] Nature Computational Science, *Peer-review criteria*. Journal peer-review policy
- [18] Journal of Open Source Software, *Minimum publishable unit and substantial scholarly effort*. JOSS editorial guidance
- [19] NASA Planetary Data System, *PDS4 Information Model Specification*. <https://pds.nasa.gov/datastandards/documents/im/>
- [20] NASA Planetary Data System, *PDS4 Processing Information Dictionary*. <https://pds.nasa.gov/datastandards/dictionaries/>
- [21] NASA Earthdata, *Applying International Metadata Standards to NASA Earth Science Data*, 2020. NASA Earthdata article
- [22] NASA Earthdata, *Instant Karma: Applying a Proven Provenance Tool to NASA*. NASA provenance project page
- [23] NASA Science Mission Directorate, *NASA Open Science Funding Opportunities*. NASA open-science funding page
- [24] NASA Science Mission Directorate, *ROSES Open Science and Data Management Plan*, updated 2025. NASA OSDMP guidance
- [25] European Space Agency Earth Observation, *EarthCODE and the Open Science Catalog*, 2025. ESA Open Science and EarthCODE overview
- [26] Great Expectations, *Data Validation Workflow and Checkpoints*. Great Expectations validation documentation
- [27] Pandera Project, *Data Validation Schemas and Checks*. <https://pandera.readthedocs.io/>
- [28] TensorFlow, *TensorFlow Data Validation*. TFDV documentation
- [29] NASA Planetary Data System, *PDS4 Validate Tool and Training*. <https://pds.nasa.gov/datastandards/training/>
- [30] NASA High Energy Astrophysics Science Archive Research Center, *FITSVERIFY: A FITS File Format-Verification Tool*. <https://heasarc.gsfc.nasa.gov/docs/software/ftools/fitsverify/>
- [31] Astropy Project, *FITS Verification*. Astropy FITS verification documentation
- [32] P. Missier, S. Woodman, H. Hiden, and P. Watson, “Provenance and data differencing for workflow reproducibility analysis,” 2014. <https://arxiv.org/abs/1406.0905>
- [33] U. Kanewala and J. M. Bieman, “Testing scientific software: A systematic literature review,” *Information and Software Technology*, vol. 56, no. 10, pp. 1219–1232, 2014.
- [34] U. Kanewala and T. Y. Chen, “Metamorphic testing: A simple yet effective approach for testing scientific software,” *Computing in Science & Engineering*, vol. 21, no. 1, pp. 66–72, 2019.
- [35] M. Li, X. Yang, J. Liu, and S. Yan, “Domain-validity-gated metamorphic testing of scientific ML surrogates,” preprint, 2026. <https://arxiv.org/abs/2606.17529>

- [36] M. Foster and S. Tregaele, “Physical-type correctness in scientific Python,” 2018. <https://arxiv.org/abs/1807.07643>
- [37] A. Chevrot, “impunity: Enforcing physical unit consistency at definition time in Python,” *Journal of Open Aviation Science*, 2023. <https://journals.open.tudelft.nl/joas/article/view/7071>
- [38] C. Wang et al., “Integrating physical units into high-performance AI-driven scientific computing,” *Nature Communications*, 2025. <https://www.nature.com/articles/s41467-025-58626-4>
- [39] NASA Lessons Learned Information System, *Coordinate Frame Conventions and Documentation*, Lesson 692. <https://llis.nasa.gov/lesson/692>
- [40] European Space Agency, *Pyxel: A Software Framework for Imaging Detector Simulation*. ESA Pyxel project page
- [41] A. Kennedy, *Types for Units-of-Measure: Theory and Practice*, 2009. https://typesatwork.imm.dtu.dk/material/TaW_Paper_TypesAtWork_Kennedy.pdf
- [42] Microsoft, *F# Units of Measure Language Reference*. <https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/units-of-measure>
- [43] AdaCore, *Physical Units Pass the Generic Test*, 2017. <https://www.adacore.com/blog/physical-units-pass-the-generic-test>
- [44] M. Pusz et al., *mp-units: A Physical Quantities and Units Library for Modern C++*. <https://mpusz.github.io/mp-units/>
- [45] ISO C++ Working Group, *P3045R4: Quantities and Units Library*, 2024. <https://isocpp.org/files/papers/P3045R4.html>
- [46] S. Foster and B. Wolff, “Automated reasoning for physical quantities, units, and measurements in Isabelle/HOL,” 2023. <https://arxiv.org/abs/2302.07629>
- [47] Astropy Project, *Unit-Aware Type Annotations*. https://docs.astropy.org/en/latest/units/type_hints.html
- [48] Python Typing Council, *Typing Specification: Annotated Metadata*. <https://typing.python.org/en/latest/spec/qualifiers.html>
- [49] NumPy Project, *NumPy Typing and Shape-Typing Limitations*, 2026. <https://numpy.org/devdocs/release/2.5.0-notes.html>
- [50] S. Kate, M. Chinn, H. Choi, X. Zhang, and S. Elbaum, “PHYSFRAME: Type checking physical frames of reference for robotic systems,” in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021. https://hongjun9.github.io/pubs/FSE21_Physframe.pdf
- [51] L. Koppel and S. L. Waslander, “Manifold geometry with fast automatic derivatives and coordinate frame semantics checking in C++,” 2018. <https://arxiv.org/abs/1805.01810>